

claude-windows / 05c32f87-3bfb-4a7f-861c-0050ac4169ac

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\05c32f87-3bfb-4a7f-861c-0050ac4169ac\subagents\workflows\wf_a8f43370-4cb\agent-ae6d36e2a5d15c57e.jsonl`  
- SHA-256: `37f2b9363272ef363217ff08d2b7a741bb360d2200ed5e7ad4a0dadbe0f24063`  
- Source modified: `2026-07-07T15:26:47+00:00`  
- Imported at: `2026-07-09T08:32:24+00:00`  
- project: `wf_a8f43370-4cb`  
- session_id: `05c32f87-3bfb-4a7f-861c-0050ac4169ac`
```

Transcript

1. user (2026-07-07T15:18:57.522Z)

You are reviewing a two-repo bug fix BEFORE it merges into a HOSTED PRODUCTION Cloud Run control-plane.

THE BUG: On the hosted control-plane, POST /api/process with compute="local" ran the in-process segmenter ON THE CONTROL-PLANE, which bakes NO detector weights (only the worker Cloud Run Job GCSFuse-mounts weights at /gcs/models/...). It died deep with a cryptic "WASB weights not found at /opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar".
[COMPUTE] requested=local actual=in_process dispatched=None status=failed.

THE FIX (2 parts):

```
- BACKEND (repo undefined, branch undefined, PR #undefined):  
  backend/orchestration.py adds _WEIGHTS_BACKED_SEGMENTERS = {wasb_hybrid, tracknet_hybrid,  
    fusion_hybrid} and _local_detector_weights_present(cfg, seg_name). In _maybe_dispatch_remote  
  the  
    choice=="local" branch now fails fast via _failed(msg, False) when the segmenter is weights-  
  backed  
    AND _cloud_available(cfg) AND local weights are not readable; otherwise returns None (in-  
  process as  
    before). Plus 4 tests in tests/test_api_cloud_dispatch.py.  
- SPA (repo undefined, branch undefined, PR #undefined):  
  web/src/components/IngestPanel.tsx compute picker defaults to "cloud" when  
  capabilities.deployment.compute_target=="cloud_run" (else keeps the free "local" default).  
Plus 1 test.
```

AUTHORITATIVE CODE ? the working trees may NOT contain the change; read these snapshot files:

```
- undefined/backend.diff (backend commit diff)  
- undefined/orchestration_new.py (FULL post-change backend/orchestration.py)  
- undefined/test_api_cloud_dispatch_new.py (FULL post-change tests)  
- undefined/spa.diff (SPA commit diff)  
- undefined/IngestPanel_new.tsx (FULL post-change IngestPanel.tsx)
```

You MAY read other repo files for ground truth, e.g. under undefined:

```
  backend/pipeline/detectors/native_wasb_runner.py, wasb_runner.py, tracknet_runner.py;  
  backend/pipeline/segmenters/{wasb_hybrid,tracknet_hybrid,fusion_hybrid,trajectory_hybrid}.py;  
  backend/config/models.py; backend/main.py (the /api/capabilities handler ~line 364-410);  
and under undefined: web/src/api/types.ts, web/src/api/client.ts,  
web/src/components/IngestPanel.test.tsx.
```

You may also run: `gh pr diff undefined -R Khelsutra/badminton-highlight-indexer` and `gh pr diff undefined -R avidullu/khelsutra`.

Report ONLY real, concrete defects that would cause wrong behavior, a bad/incorrect merge, a regression, or a materially worse UX ? each with a SPECIFIC failure scenario (concrete config/inputs

? the wrong result). Verify claims against the actual code before reporting; do not invent style nits.

If nothing real exists in your dimension, return an empty findings array.

Adversarially VERIFY this finding from the 'weights-resolution-fidelity' review. Your job is to REFUTE it if at all possible: read the ACTUAL authoritative code (the snapshot files above and any repo files needed) and determine whether the described failure REALLY occurs against the real code as written. Default to REFUTED unless you can reproduce it concretely. Beware findings that assume a code path that does not exist, or ignore an earlier guard.

FINDING:

- severity: low
- category: weights-resolution-fidelity
- location: backend/orchestration.py ? _local_detector_weights_present() (whole function: it consults only env WASB_WEIGHTS + indexing.<det>.weights_path and never reads indexing.detector_impl / RALLY_DETECTOR_IMPL)
- claim: The guard hard-codes the native-runner weights resolution and ignores detector_impl, so it does not mirror the two other runners: the stub runner (needs NO weights; healthcheck returns True unconditionally) and the WSL runner (weights live INSIDE WSL and it ignores WASB_WEIGHTS). This yields a false-BLOCK of a compute=local run that would actually succeed.
- failure_scenario: (a) Stub: a cloud-capable box (compute_target=cloud_run ? _cloud_available True) configured with indexing.detector_impl="stub" (or RALLY_DETECTOR_IMPL=stub ? a CI/demo/staging box that mimics hosted config to avoid GPU cost). POST /api/process segmenter=wasb_hybrid compute=local ? StubDetectorRunner.healthcheck() returns True and run_predict writes a deterministic CSV with no weights/GPU/WSL, so the run WOULD succeed; but _local_detector_weights_present sees no wasb weights on disk ? returns False ? the guard fast-fails "this hosted server can't run detection locally", wrongly blocking a working run (and pushing the user to paid cloud). (b) WSL divergence: a cloud-wired Windows+WSL box with detector_impl=auto and WASB weights at the default WSL path ~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar (present inside WSL, where WasbRunner reads them): the local run would succeed, but the guard expanduser's that path against the Windows HOST home (C:\Users\<user>\models\WASB-SBDT\...) where it does not exist ? returns False ? false-block. Both require non-default configs (stub+cloud, or WSL+cloud-wired), hence low severity.
- suggested_fix: Consult the impl the run will actually use: short-circuit to True (not-guardrailed) when RALLY_DETECTOR_IMPL/indexing.detector_impl == "stub"; and either scope the guard to detector_impl=="native" (the only impl whose resolution it faithfully mirrors) or resolve weights the way the selected runner does (WSL runner = config weights_path only, checked inside WSL), so the guard mirrors the runner that will execute rather than assuming the native path.

Return CONFIRMED only with a concrete repro grounded in the real code; else REFUTED with the reason.

2. assistant (2026-07-07T15:19:05.112Z)

I'll start by locating the authoritative snapshot files and the real repo code, since the task has "undefined" placeholders for paths.

3. user (2026-07-07T15:19:07.502Z)

total 108

```
drwxr-xr-x 1 avidu 197609  0 Jul  7 19:57 .
drwxr-xr-x 1 avidu 197609  0 Jul  7 20:24 ..
drwxr-xr-x 1 avidu 197609  0 Jun 23 09:53 Annotation Setup
-rw-r--r-- 1 avidu 197609 905 Jul  2 21:20 AUDIT_REPORT_khelsutra-guru_2026-07-02.md
drwxr-xr-x 1 avidu 197609  0 Jul  7 20:49 badminton-highlight-indexer
-rw-r--r-- 1 avidu 197609 1222 Jul  2 21:35 bash.exe.stackdump
drwxr-xr-x 1 avidu 197609  0 Jul  7 20:34 bhi-wt-worker-reval
drwxr-xr-x 1 avidu 197609  0 Jul  5 12:57 khelsutra
drwxr-xr-x 1 avidu 197609  0 Jul  5 18:47 khelsutra-screencast
drwxr-xr-x 1 avidu 197609  0 Jul  5 13:44 marketing
drwxr-xr-x 1 avidu 197609  0 Jul  5 13:43 pr-158-screenshots
drwxr-xr-x 1 avidu 197609  0 Jun 22 15:03 rally-annotator
```

```

drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 sports-data-collector
drwxr-xr-x 1 avidu 197609    0 Jul  2 19:46 sports-obsreport
---KHELSUTRA---
total 62
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:57 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609    0 Jun 22 11:12 .claude
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:38 .git
-rw-r--r-- 1 avidu 197609 297 Jun 20 23:27 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jun 20 12:43 .github
-rw-r--r-- 1 avidu 197609 323 Jun 22 14:57 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jul  2 21:13 deploy
drwxr-xr-x 1 avidu 197609    0 Jul  2 21:13 docs
drwxr-xr-x 1 avidu 197609    0 Jun 21 23:55 locales
-rw-r--r-- 1 avidu 197609 9773 Jul  2 21:13 NEXT_STEPS.md
drwxr-xr-x 1 avidu 197609    0 Jun 21 15:38 node_modules
-rw-r--r-- 1 avidu 197609 1469 Jun 23 00:19 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 23 00:19 site
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:45 web

```

4. user (2026-07-07T15:19:11.220Z)

```

---
---spa---
---ingest---

```

5. assistant (2026-07-07T15:19:15.113Z)

The snapshot files aren't where I looked. Let me search more broadly and check the real repo location per memory.

6. user (2026-07-07T15:19:20.799Z)

```

total 702
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:49 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609    0 Jun 25 23:31 .claude
-rw-r--r-- 2 avidu 197609 53248 Jul  7 20:49 .coverage
-rw-r--r-- 1 avidu 197609  611 Jun 11 10:30 .coveragerc
-rw-r--r-- 1 avidu 197609   69 Jun 10 23:25 .env
-rw-r--r-- 1 avidu 197609 1059 Jul  7 18:29 .gcloudignore
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:44 .git
-rw-r--r-- 1 avidu 197609  947 Jun 25 16:24 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jul  2 08:13 .github
-rw-r--r-- 1 avidu 197609 1998 Jul  2 08:13 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:42 .mypy_cache
drwxr-xr-x 1 avidu 197609    0 May 19 15:58 .pytest_cache
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:32 .ruff_cache
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 .vscode
drwxr-xr-x 1 avidu 197609    0 Jul  7 10:27 __pycache__
drwxr-xr-x 1 avidu 197609    0 Jul  4 00:22 artifacts
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:56 backend
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 backend.db
-rw-r--r-- 1 avidu 197609  6619 Jul  2 11:50 CLAUDE.md
-rw-r--r-- 1 avidu 197609 2735 Jun 16 01:52 config.json
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 deploy
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:43 docs
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 eval_baselines
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 indexer.db
-rw-r--r-- 1 avidu 197609 1999 Jun 25 16:24 mypy.ini
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:48 output
-rw-r--r-- 1 avidu 197609  5413 Jul  4 13:58 pyproject.toml
-rw-r--r-- 1 avidu 197609   240 May 30 14:08 pytest.ini
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 rally_index.db

```

```

-rw-r--r-- 1 avidu 197609 35511 Jul  2 08:13 README.md
-rw-r--r-- 1 avidu 197609 1674 Jul  2 20:05 requirements.txt
-rw-r--r-- 1 avidu 197609  585 Jul  2 19:49 requirements-trackers.txt
-rw-r--r-- 1 avidu 197609  809 Jun 25 16:24 ruff.toml
-rwxr-xr-x 1 avidu 197609 2118 Jun 29 13:47 run_all_tests.py
-rw-r--r-- 1 avidu 197609 5739 Jun 29 13:47 run_eval.py
-rw-r--r-- 1 avidu 197609 4594 Jun 29 13:47 run_regression.py
-rwxr-xr-x 1 avidu 197609 5073 Jul  7 10:27 score_golden_gate.py
-rwxr-xr-x 1 avidu 197609 7617 Jul  7 03:32 score_local.py
---AppData scratch---
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/orchestration_new.py
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/orchestration_post.py
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/extract_orchestration.py
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/backend.diff
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/spa.diff
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr35.diff
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr37.diff
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr38.diff
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr39.diff
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr50.diff
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr51.diff
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr52.diff
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr53.diff

```

7. user (2026-07-07T15:19:22.066Z)

```

/c/Users/avidu/AppData/Local/Temp/backend.diff
/c/Users/avidu/AppData/Local/Temp/spa.diff

```

8. assistant (2026-07-07T15:19:29.060Z)

Found the snapshots. Let me read the authoritative backend orchestration file and the diff.

9. user (2026-07-07T15:19:30.399Z)

```

1      commit 4565c7436ccf0b4e30f68f01efe6082eff371a5d
2      Author: Avi Dullu <avi.dullu@gmail.com>
3      Date:   Tue Jul 7 19:56:22 2026 +0530
4
5          fix(serving): fail fast when compute=local forces weightless in-process detection
6
7          A hosted cloud-serving control-plane bakes NO detector weights into its image
8          (only the worker Cloud Run Job GCSFuse-mounts them at /gcs/models/...). So a
9          POST /api/process with compute="local" for a weights-backed trajectory detector
10         (wasb_hybrid / tracknet_hybrid / fusion_hybrid) returned None from
11         _maybe_dispatch_remote and ran the in-process segmenter ON THE CONTROL-PLANE,
12         which then died deep in the healthcheck with the confusing
13         "WASB weights not found at /opt/models/WASB-SBDT/..." (observed live 2026-07-07,
14         job f29d41d0?: [COMPUTE] requested=local actual=in_process ... status=failed).
15
16         Guardrail: on a cloud-serving-capable server (_cloud_available) whose local

```

```

17     weights for the requested detector aren't readable, a forced compute="local"
18     now fails FAST with an actionable message ("this hosted server can't run
19     detection locally ... Choose 'Run in cloud'.") ? mirroring the existing
20     compute="cloud"-not-available reject ? instead of the silent in-process trap.
21     A truly-local box, or a cloud-wired box that actually has the weights on disk,
22     is unaffected (the weights-present check is the escape hatch); a weight-free
23     segmenter (motion) is out of scope. Honest provenance: path=not_run,
24     requested=local, dispatched=False.
25
26     Covered by tests/test_api_cloud_dispatch.py (fail-fast, weights-present escape
27     hatch, motion unaffected, pure-local not guardrailed).
28
29     Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
30
31     diff --git a/backend/orchestration.py b/backend/orchestration.py
32     index 5e9f510..bfe39f5 100644
33     --- a/backend/orchestration.py
34     +++ b/backend/orchestration.py
35     @@ -155,6 +155,38 @@ def _ingest_remote_segments(
36         return n
37
38
39     ## Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS
40     box. A hosted
41     ## cloud-serving control-plane bakes NO detector weights into its image (only the worker
42     Cloud Run Job
43     ## GCSFuse-mounts them), so forcing compute="local" for one of these there can never
44     succeed ? the
45     ## guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
46     deep
47     ## "WASB weights not found". (Names are the registered segmenter identities; see
48     backend/pipeline/segmenters.)
49     +_WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
50     "fusion_hybrid"})
51     +
52     +
53     +def _local_detector_weights_present(cfg, seg_name: str) -> bool:
54     +    """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
55     present + readable
56     +    on THIS box. Mirrors the native/WSL runners' resolution (env override ?
57     ``indexing.<det>.weights_path``)
58     +    and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
59     that would
60     +    otherwise fail deep in the run. Fusion follows its configured substrate (WASB
61     default / TrackNet).
62     +    A non-weights segmenter returns True (nothing local to check)."""
63     +    name = (seg_name or "").strip().lower()
64     +    if name not in _WEIGHTS_BACKED_SEGMENTERS:
65     +        return True
66     +    idx = getattr(cfg, "indexing", None)
67     +    substrate = (
68     +        getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
69     +    ).strip().lower()
70     +    if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
71     "tracknet"):
72     +        path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
73     +    else: # wasb_hybrid, or fusion on the (default) WASB substrate
74     +        path = (
75     +            os.environ.get("WASB_WEIGHTS")
76     +            or getattr(getattr(idx, "wasb", None), "weights_path", "")
77     +            or ""
78     +        )
79     +    return bool(path) and os.path.exists(os.path.expanduser(path))

```

```

71     def _cloud_available(cfg) -> bool:
72         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
73 3).
74
75     @@ -446,6 +478,23 @@ def _maybe_dispatch_remote(
76         )
77         choice = ""
78         if choice == "local":
79             + # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
79 + # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
80 + # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
81 + # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail
FAST with an
82 + # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
83 + # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it
runs in-process
84 + # below exactly as before (the weights-present check is the escape hatch).
85 + if (
86 +     (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
87 +     and _cloud_available(cfg)
88 +     and not _local_detector_weights_present(cfg, seg_name)
89 + ):
90 +     return _failed(
91 +         f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
92 +         "live only on the cloud worker, not on the control-plane. Choose 'Run
in cloud'.",
93 +         False,
94 +     )
95     return None # explicit "run on my machine" ? in-process regardless of the
server's target
96     if choice == "cloud":
97         if not _cloud_available(cfg):
98             diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
99             index 8382234..7993b7f 100644
100             --- a/tests/test_api_cloud_dispatch.py
101             +++ b/tests/test_api_cloud_dispatch.py
102             @@ -721,6 +721,114 @@ def
test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypa
103                 assert all(_meta(s)["source"] == "cloud_run" for s in segs)
104
105
106     +## --- hosted control-plane guardrail: compute='local' for a weights-backed detector
-----
107     +## A hosted cloud-serving control-plane bakes NO detector weights (only the worker Cloud
Run Job mounts
108     +## them), so a forced compute='local' for wasb/tracknet/fusion can't run in-process
there. It must fail
109     +## FAST with an actionable "run in the cloud" message ? not fall through to the in-
process segmenter
110     +## that dies deep with a cryptic "WASB weights not found" (observed live: job f29d41d0?,
2026-07-07).
111     +
112     +
113     +def test_local_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
114     +     """compute='local' + a weights-backed detector on a cloud_run server with NO local
weights ?
115     +     a clear pre-dispatch failure (never dispatched, never the cryptic deep 'weights not
found')."""
116     +     _db, _bucket, _mp = cloud_env
117     +     monkeypatch.delenv("WASB_WEIGHTS", raising=False)

```

```

118 +     m.global_config.indexing.wasb.weights_path =
119 +         "/nonexistent/wasb_badminton_best.pth.tar"
120 +     fake = _FakeCloudClient()
121 +     _inject_fake(monkeypatch, fake)
122 +
123 +     out = m._execute_processing(
124 +         m.ProcessRequest(
125 +             video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
126 +         )
127 +     )
128 +     assert out["status"] == "failed"
129 +     assert "can't run detection locally" in out["message"]
130 +     assert "Run in cloud" in out["message"]
131 +     assert "weights not found" not in out["message"] # NOT the cryptic deep failure
132 +     assert fake.calls == [] # never dispatched
133 +     # Honest provenance: requested local, ran NOWHERE (pre-dispatch reject).
134 +     assert out["compute"]["requested"] == "local"
135 +     assert out["compute"]["dispatched"] is False
136 +     assert out["compute"]["path"] == "not_run"
137 +
138 + def test_local_on_cloud_server_with_weights_present_runs_in_process(
139 +     cloud_env, tmp_path, monkeypatch
140 + ):
141 +     """Escape hatch: the guardrail must NOT over-block ? on a cloud-wired box that
142 +     ACTUALLY has the
143 +     detector weights on disk, compute='local' still runs the in-process segmenter (no
144 +     dispatch)."""
145 +     _db, _bucket, _mp = cloud_env
146 +     weights = tmp_path / "wasb_badminton_best.pth.tar"
147 +     weights.write_bytes(b"WEIGHTS")
148 +     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
149 +     m.global_config.indexing.wasb.weights_path = str(weights)
150 +     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
151 +     fake = _FakeCloudClient()
152 +     _inject_fake(monkeypatch, fake)
153 +
154 +     out = m._execute_processing(
155 +         m.ProcessRequest(
156 +             video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
157 +         )
158 +     )
159 +     assert out["status"] == "success"
160 +     assert fake.calls == [] # in-process, not dispatched
161 +     assert out["compute"]["path"] == "in_process"
162 +
163 + def test_local_non_weights_segmenter_on_cloud_server_still_runs_in_process(
164 +     cloud_env, monkeypatch
165 + ):
166 +     """The guardrail is scoped to weights-backed detectors: a weight-free segmenter
167 +     (motion) with
168 +     compute='local' on a cloud server is unaffected ? it runs in-process, never fails
169 +     fast."""
170 +     _db, _bucket, _mp = cloud_env
171 +     m.global_config.indexing.wasb.weights_path =
172 +         "/nonexistent/should_not_matter.pth.tar"
173 +     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
174 +     fake = _FakeCloudClient()
175 +     _inject_fake(monkeypatch, fake)
176 +
177 +     out = m._execute_processing(
178 +         m.ProcessRequest(
179 +             video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
180 +         )
181 +     )

```

```

177 + )
178 + assert out["status"] == "success"
179 + assert fake.calls == []
180 + assert out["compute"]["path"] == "in_process"
181 +
182 +
183 +def test_local_weights_backed_on_pure_local_server_is_not_guardrailed(
184 +     tmp_path, monkeypatch
185 +):
186 +     """The guardrail only fires on a cloud-capable server. On a pure-local server (no
cloud), a
187 +     weights-backed compute='local' request falls through to the in-process segmenter as
before ? the
188 +     'run in the cloud' message would be nonsense there (no cloud to fall back to)."""
189 +     db = Database(str(tmp_path / "t.db")) # noqa: F841
190 +     monkeypatch.setattr(m, "db_instance", db)
191 +     cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}}) # noqa:
F841
192 +     monkeypatch.setattr(m, "global_config", cfg)
193 +     monkeypatch.setattr(
194 +         m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
195 +     )
196 +     monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
197 +     monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
198 +     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
199 +     cfg.indexing.wasb.weights_path = "/nonexistent/wasb.pth.tar"
200 +     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
201 +     video = tmp_path / "v.mp4"
202 +     video.write_bytes(b"hashable-bytes")
203 +
204 +     out = m._execute_processing(
205 +         m.ProcessRequest(
206 +             video_path=str(video), segmenter="wasb_hybrid", compute="local"
207 +         )
208 +     )
209 +     # Not the hosted fast-fail ? it ran in-process (the local box owns its own weights
story).
210 +     assert out["status"] == "success"
211 +     assert out["compute"]["path"] == "in_process"
212 +
213 +
214 +     # --- compute-path PROVENANCE (the validation method: prove which backend ran)
-----
215
216
217

```

10. user (2026-07-07T15:19:30.465Z)

```

1     """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3     The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4     Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5     and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6     Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8     SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9     monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10     ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through

```

```

11     the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12     bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13     ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14     This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15     Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16     ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17     Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18     objects and need no routing.
19     """
20
21     import csv
22     import json
23     import logging
24     import math
25     import os
26     import re
27     import tempfile
28     import threading
29     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30     from urllib.parse import quote
31
32     from backend import storage
33     from backend.config import AppConfig, StitchingConfig
34     from backend.pipeline.segmenters.base import segmenter_registry
35     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
36     from backend.pipeline.validators.base import registry
37     from backend.results import Failure
38     from backend.utils.paths import output_base, safe_output_filename
39
40     if (
41         TYPE_CHECKING
42     ): # runtime import would be circular (main imports us for the re-export)
43         from backend.main import ProcessRequest # noqa: F401
44
45     logger = logging.getLogger(__name__)
46
47     # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
48     # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
49     # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
50     # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
51     # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
52     # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
53     _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
54
55
56     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
57         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
58
59         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
60         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
61         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
62         """

```

```

63         if segments:
64             db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
65         else:
66             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
67
68
69     def _validation_failure_message(failures: List[Any]) -> str:
70         details = []
71         for failure in failures:
72             name = getattr(failure, "validator_name", None) or "validation"
73             message = (getattr(failure, "message", None) or "").strip()
74             details.append(f"{name}: {message}" if message else str(name))
75         if not details:
76             return "Video failed validation checks."
77         return "Video failed validation: " + "; ".join(details)
78
79
80     def _report_config_for_input(config: Any, raw_input: str) -> Any:
81         """Return report config that keeps remote-input reports out of disposable
82         scratch."""
83         storage_config = (
84             config.get("storage")
85             if isinstance(config, dict)
86             else getattr(config, "storage", None)
87         )
88         try:
89             if storage.input_is_local(raw_input, storage_config):
90                 return config
91         except ValueError:
92             return config
93         try:
94             cfg = (
95                 config.model_dump(mode="json")
96                 if hasattr(config, "model_dump")
97                 else dict(config or {})
98             )
99         except Exception:
100             cfg = {}
101         report = dict(cfg.get("report") or {})
102         if report.get("output_dir"):
103             return config
104         report["output_dir"] = os.path.join(output_base(config), "reports")
105         cfg["report"] = report
106         return cfg
107
108     def _ingest_remote_segments(
109         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110     ) -> int:
111         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
112         the bucket and
113         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
114         call ? so
115         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
116         were produced.
117         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
118         pattern (cloud
119         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
120         malformed row is
121         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
122         temp dir is
123         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
124         a job failure
125         and prunes any partial rows (destroy-AFTER-confirm)."""
126         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"

```

```

120     ref = storage.parse_uri(uri)
121     n = 0
122     with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
123         local = storage.fetch(
124             uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
125         )
126         with open(local, newline="", encoding="utf-8") as f:
127             for row in csv.DictReader(f):
128                 if n >= max_rows:
129                     logger.warning(
130                         "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
131                         max_rows,
132                         uri,
133                     )
134                     break
135                 try:
136                     start, end = float(row["start"]), float(row["end"])
137                 except (KeyError, TypeError, ValueError):
138                     continue
139                 if not (math.isfinite(start) and math.isfinite(end)):
140                     continue # reject inf/nan times rather than persist a junk rally
141                 meta: Dict[str, Any] = {
142                     "source": "cloud_run",
143                     "ending_reason": (row.get("ending_reason") or "").strip(),
144                 }
145                 sc = (row.get("shot_count") or "").strip()
146                 if sc:
147                     try:
148                         meta["shot_count"] = int(
149                             float(sc)
150                         ) # OverflowError on 'inf'/'1e400'
151                     except (ValueError, OverflowError):
152                         meta["shot_count"] = sc
153                 db.add_play_segment(video_id, start, end, metadata=meta)
154                 n += 1
155     return n
156
157
158     # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
159     # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
160     # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
161     # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
162     # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
163     _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
164
165
166     def _local_detector_weights_present(cfg, seg_name: str) -> bool:
167         """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
168         on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
169         and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
170         otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
171         A non-weights segmenter returns True (nothing local to check)."""
172         name = (seg_name or "").strip().lower()
173         if name not in _WEIGHTS_BACKED_SEGMENTERS:

```

```

174         return True
175     idx = getattr(cfg, "indexing", None)
176     substrate = (
177         getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
178     ).strip().lower()
179     if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
180         path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
181     else: # wasb_hybrid, or fusion on the (default) WASB substrate
182         path = (
183             os.environ.get("WASB_WEIGHTS")
184             or getattr(getattr(idx, "wasb", None), "weights_path", "")
185             or ""
186         )
187     return bool(path) and os.path.exists(os.path.expanduser(path))
188
189
190 def _cloud_available(cfg) -> bool:
191     """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
192
193     True when the control plane is wired for Cloud Run: either the configured default
target is
194     already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
195     (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
196     are present so a forced per-request override can actually dispatch + collect a
result.
197
198     Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
199     deployment = getattr(cfg, "deployment", None)
200     if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201         return True
202     has_job = bool(
203         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204     )
205     out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206     return bool(has_job and out_root)
207
208
209 def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210     """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
211     with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
212     (queue/cold-start), so the very first tick already shows movement."""
213     minutes = int(elapsed_sec) // 60
214     if minutes < 1:
215         return f"segmenting ({seg_name}) on cloud GPU ? starting up"
216     return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
217
218
219
220 def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221     """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222     object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223     deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224     ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
225     if getattr(input_ref, "backend", "") != "gcs":
226         return None

```

```

227     sc = _storage_settings(cfg)
228     if not sc:
229         return None
230     try:
231         st = storage.get_storage("gcs", config=sc).stat(input_ref)
232         return getattr(st, "md5", None) or None
233     except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
234         logger.info(
235             "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236             getattr(input_ref, "uri", input_ref),
237             exc_info=True,
238         )
239     return None
240
241
242 def _cached_process_payload(
243     video_id: str, validation_results, segments, requested_compute
244 ) -> Dict[str, Any]:
245     """The /api/process result for a video whose detection already exists ? ONE shape
246     for
247     both producers (the drain's DB-cache short-circuit and the submit-time cache,
248 #484)."""
249     return {
250         "video_id": video_id,
251         "status": "success",
252         "message": (
253             f"Reused {len(segments)} cached play segments. "
254             "Pass force=true to reprocess this video."
255         ),
256         "results": validation_results or [],
257         "play_segments": segments,
258         "cached": True,
259         "compute": {"path": "cached", "requested": requested_compute},
260     }
261
262 def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
263     """Fetch + parse one small JSON object (cloud backends require a real dst ? the
264     ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
265     decide."""
266     ref = storage.parse_uri(uri)
267     with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
268         local = storage.fetch(
269             uri, os.path.join(td, ref.name or "report.json"), config=storage_config
270         )
271         with open(local, encoding="utf-8") as f:
272             loaded = json.load(f)
273         return loaded if isinstance(loaded, dict) else None
274
275 # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
276 hits
277 # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
278 # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
279 instance
280 # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
281 _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
282 _REHYDRATE_LOCKS_GUARD = threading.Lock()
283
284 def _rehydrate_lock(video_id: str) -> threading.Lock:
285     with _REHYDRATE_LOCKS_GUARD:
286         return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())

```



```

342         video_id,
343         existing.get("validation_results", []),
344         segs,
345         requested_compute,
346     )
347     sc = _storage_settings(cfg)
348     if not sc or (sc.get("backend") or "local") == "local":
349         return None
350     output_root = (sc.get("output_root") or "").rstrip("/")
351     if not output_root:
352         return None
353     outputs_uri = f"{output_root}/outputs/{video_id}"
354     report_uri = f"{outputs_uri}/report.json"
355     if not storage.exists(report_uri, config=sc):
356         return None
357     report = _read_bucket_json(report_uri, sc)
358     if not report or report.get("status") != "success":
359         return None
360     if not video_path:
361         # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
362         # carries the worker's --video-uri (P3); older reports fall back to the
363         # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
364         # recover, and the next same-md5 submit re-binds the truthful path via
365         # add_video's upsert.
366         video_path = str(report.get("video_uri") or "") or _first_object_under(
367             f"{output_root}/videos/{video_id}/", sc
368         )
369     play_wm, cand_wm = db.segment_watermarks(video_id)
370     db.add_video(video_id, video_path, "processed", [])
371     n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
372     if n <= 0:
373         return None
374     segments = [
375         s
376         for s in db.query_play_segments(video_id, {})
377         if int(s.get("id", 0)) > play_wm
378     ]
379     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
380     if not segments:
381         return None
382     logger.info(
383         "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
384         n,
385         video_id,
386         outputs_uri,
387     )
388     return _cached_process_payload(video_id, [], segments, requested_compute)
389 except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
390     logger.warning(
391         "[API] submit-time cache check failed for %s ? dispatching normally",
392         video_id,
393         exc_info=True,
394     )
395     return None
396
397
398 def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
399     """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
400     try:
401         return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
402 except Exception: # noqa: BLE001 ? recovery nicety, never a failure
403     return ""

```

```

404
405
406 _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
407
408
409 def read_path_rehydrate(db, cfg, video_id: str) -> bool:
410     """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
411     nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
412     so
413     ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
414     DB
415     has the video's rows now (re-query). Non-md5 ids return False without touching the
416     bucket; every failure degrades to False (the read just reports what the DB has)."""
417     vid = (video_id or "").strip().lower()
418     if not _MD5_HEX_RE.fullmatch(vid):
419         return False
420     if not probe_process_cache(db, cfg, vid):
421         return False
422     return cached_process_result(db, cfg, vid, "") is not None
423
424 def _maybe_dispatch_remote(
425     cfg,
426     db,
427     req: "ProcessRequest",
428     video_id: str,
429     seg_name: str,
430     val_results,
431     play_wm: int,
432     cand_wm: int,
433     notify,
434 ) -> Optional[Tuple[Dict[str, Any], bool]]:
435     """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
436     ``deployment.compute_target
437     == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
438     intervals into
439     THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
440     Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
441     the cloud job
442     actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
443     observability report
444     doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
445     process segmenter
446     (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
447     non-``cloud_run`` target, so
448     the in-process path stays byte-identical)."""
449     from backend.compute import ComputeJobSpec, get_compute_backend
450
451     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
452         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
453         (id > play_wm);
454         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
455         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
456         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
457         only when the
458         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
459         rc / ingest
460         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
461         records the
462         # intended backend, `requested` the picker choice, `dispatched` whether the
463         remote job ran.
464         return (
465             {
466                 "video_id": video_id,

```

```

456         "status": "failed",
457         "message": msg,
458         "results": [r.model_dump() for r in val_results],
459         "play_segments": [],
460         "compute": {
461             "path": "cloud_run" if ran else "not_run",
462             "requested": req.compute,
463             "target": "cloud_run",
464             "dispatched": ran,
465         },
466     },
467     ran,
468 )
469
470     # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
471     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
472     # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
473     choice = (req.compute or "").strip().lower()
474     if choice and choice not in ("local", "cloud"):
475         logger.warning(
476             "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
477             req.compute,
478         )
479     choice = ""
480     if choice == "local":
481         # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
482         # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
483         # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
484         # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
485         # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
486         # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
487         # below exactly as before (the weights-present check is the escape hatch).
488         if (
489             (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
490             and _cloud_available(cfg)
491             and not _local_detector_weights_present(cfg, seg_name)
492         ):
493             return _failed(
494                 f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
495                 "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'." ,
496                 False,
497             )
498             return None # explicit "run on my machine" ? in-process regardless of the
server's target
499         if choice == "cloud":
500             if not _cloud_available(cfg):
501                 return _failed(
502                     "cloud-serving was requested but this server isn't configured for it "
503                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
504                     False,
505                 )
506             compute = get_compute_backend(cfg, target_override="cloud_run")

```

```

507         else:
508             compute = get_compute_backend(
509                 cfg
510             ) # server default (default-OFF unless cloud_run)
511         if compute is None:
512             return None # default-OFF ? run the in-process segmenter below (unchanged)
513
514         storage_cfg = getattr(cfg, "storage", None)
515
516         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
517         # to this box.
518         try:
519             input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
520         except ValueError as e:
521             return _failed(
522                 f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
523             )
524         if input_ref.backend == "local":
525             # A browser upload lands on THIS box's local disk; the Cloud Run job (a
526             # different machine)
527             # can't read it. Stage it UP to the input bucket first, then dispatch from there
528             (#461). Only
529             # possible when a CLOUD input_root is configured; without one we genuinely can't
530             stage ? fail.
531             # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
532             URI below is
533             # built by joining under input_root, and storage.put routes by
534             parse_uri(staged_uri).backend ?
535             # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
536             at its
537             # default, while a bare/local-looking root can't stage no matter what
538             input_backend says
539             # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
540             # an empty root
541             # gives us nothing to build the staged URI under).
542             input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
543             try:
544                 root_backend = (
545                     storage.parse_uri(input_root).backend if input_root else "local"
546                 )
547             except ValueError: # unsupported scheme in input_root ? nowhere real to stage
548                 to
549                 root_backend = "local"
550             if root_backend == "local":
551                 return _failed(
552                     "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
553                     storage.input_root to "
554                     "stage local uploads into; a path local to this machine can't be read by
555                     the Cloud "
556                     "Run job.",
557                     False,
558                 )
559             local_src = input_ref.key # resolved local path of the upload
560             staged_uri =
561             f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
562             try:
563                 notify("staging upload to bucket")
564                 # Thread the active storage settings so backend-specific config survives (S3
565                 endpoint_url/
566                 # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
567                 default client.
568                 storage.put(
569                     local_src,
570                     staged_uri,
571                     config={

```

```

557         storage_cfg.model_dump() if storage_cfg is not None else None
558     ),
559 )
560 except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
561     logger.error(
562         "[API] cloud-serving: failed to stage %s -> %s: %s",
563         local_src,
564         staged_uri,
565         e,
566         exc_info=True,
567     )
568     return _failed(
569         f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
570         False,
571     )
572     # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
573     # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
574     # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
575     req.video_path = staged_uri
576     input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
577     db.add_video(
578         video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
579     )
580     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
581     if not out_root:
582         return _failed(
583             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
584             "? that's where the Cloud Run job writes the result.",
585             False,
586         )
587
588     notify("dispatching (cloud_run)")
589     logger.info(
590         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
591         req.video_path,
592         out_root,
593     )
594     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
595     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
596     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
597     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
598     overrides = []
599     idx_cfg = getattr(cfg, "indexing", None)
600     sk = getattr(idx_cfg, "skip_ai_handoff", None)
601     if sk is not None:
602         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
603     # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
604     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
605     # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
606     # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
607     # cuda-only (validator-enforced) and requires a worker image that understands

```

```

--decode-backend
608     # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
609     wasb_cfg = getattr(idx_cfg, "wasb", None)
610     if wasb_cfg is not None:
611         decode_backend = getattr(wasb_cfg, "decode_backend", None)
612         if decode_backend:
613             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
614         batch_size = getattr(wasb_cfg, "batch_size", None)
615         if batch_size:
616             overrides.append(f"indexing.wasb.batch_size={batch_size}")
617         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
618         if prefetch:
619             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
620     spec = ComputeJobSpec(
621         video_uri=req.video_path,
622         out_uri=out_root,
623         segmenter=seg_name,
624         config_overrides=tuple(overrides),
625     )
626     try:
627         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
628         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
629         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
630         # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
631         result = compute.run_infer(
632             spec,
633             on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
634         )
635     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
636         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
637         return _failed(f"cloud dispatch error: {e}", True)
638     if result.returncode != 0:
639         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
640     if result.video_id and result.video_id != video_id:
641         # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
642         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
643         logger.warning(
644             "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
645             "and the worker hash; ingesting under the local id.",
646             result.video_id,
647             video_id,
648         )
649
650     notify("ingesting (cloud_run)")
651     storage_dict = (
652         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
653         if hasattr(storage_cfg, "model_dump")
654         else {}
655     )
656     try:
657         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
658     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
659         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
660         return _failed(
661             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
662             True,

```

```

663         )
664
665         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
666         segments = [
667             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
668         ]
669         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
670         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
671         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
672         provenance = {
673             "path": "cloud_run",
674             "requested": req.compute,
675             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
676             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
677             "execution": result.execution,
678             "outputs_uri": result.outputs_uri,
679         }
680         logger.info(
681             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
682             result.execution,
683             provenance["job"],
684             provenance["region"],
685             result.outputs_uri,
686             video_id,
687         )
688         return (
689             {
690                 "video_id": video_id,
691                 "status": "success",
692                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
693                 "results": [r.model_dump() for r in val_results],
694                 "play_segments": segments,
695                 "compute": provenance,
696             },
697             True,
698         )
699
700
701     def _execute_processing(
702         config=None, db=None, req=None, progress=None
703     ) -> Dict[str, Any]:
704         """The full hash ? validate ? segment ? report pipeline for one video.
705
706         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
707         module-level ``global_config``/``db_instance`` (backward-compatible with existing
708         tests that pass only ``req``). Route handlers always pass explicit params; older
709         test code that calls ``_execute_processing(req)`` still works.
710
711         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
712         Raises on unexpected internal errors ? the caller records those as a job failure
713         """
714         import backend.main as _main # call-time seam: main's globals/model stay patchable
715
716         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
717         # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
718         if config is not None and not isinstance(config, AppConfig):
719             # Old signature: _execute_processing(req, progress=None)
720             if req is None and db is not None and isinstance(db, _main.ProcessRequest):

```

```

721         req = db
722         db = None
723         elif req is None and isinstance(config, _main.ProcessRequest):
724             req = config
725             config = None
726     # Resolve config/db from module globals when not passed explicitly
727     if config is None:
728         config = _main.global_config
729     if db is None:
730         db = _main.db_instance
731
732     notify = progress or (lambda stage: None)
733
734     notify("hashing")
735     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
736     # (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
737     # original # req.video_path (the source URI) is kept for the DB record + report; only the file-
738     # reading # consumers (hash / validators / segmenter) use the resolved local path.
739     local_video = ""
740     try:
741         local_video = storage.acquire_local_input(
742             req.video_path, getattr(config, "storage", None)
743         )
744         video_id = _main.compute_video_id(local_video)
745     except Exception:
746         storage.cleanup_acquired_local_input(
747             req.video_path, local_video, getattr(config, "storage", None)
748         )
749         raise
750     try:
751         existing_video = db.get_video(video_id)
752     except Exception:
753         storage.cleanup_acquired_local_input(
754             req.video_path, local_video, getattr(config, "storage", None)
755         )
756         raise
757     was_reingest = existing_video is not None
758
759     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
760     default_seg = getattr(
761         getattr(config, "indexing", None), "default_segmenter", "motion"
762     )
763     seg_name = req.segmenter or default_seg
764     segmenter_actual = None
765     segmentation_ran = False
766     val_results: List[Any] = []
767     val_context: Dict[str, Any] = {}
768     # Compute-path PROVENANCE (the validation method): which backend actually ran the
769     # "cloud_run" by # "in_process" once the local segmenter is reached, set on the response as
770     # # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
771     # fail).
772     compute_actual: Optional[str] = None
773     response: Optional[Dict[str, Any]] = None
774     try:
775         if (
776             existing_video
777             and existing_video.get("status") == "processed"
778             and not req.force
779         ):
780             cached_segments = db.query_play_segments(video_id, {})

```

```

780         if cached_segments:
781             logger.info(
782                 "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
783                 video_id,
784             )
785             response = _cached_process_payload(
786                 video_id,
787                 existing_video.get("validation_results", []),
788                 cached_segments,
789                 getattr(req, "compute", None),
790             )
791             return response
792
793     # 1. Run pluggable validators (with-context variant surfaces fprobe_meta for
the report)
794     notify("validating")
795     logger.info(f"Running validations for {req.video_path}...")
796     val_results, val_context = registry.run_all_with_context(local_video, config)
797     failures = [r for r in val_results if not r.passed]
798
799     if failures:
800         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
801         # status; it no longer destroys the video's prior good segments.
802         db.add_video(
803             video_id,
804             req.video_path,
805             "failed",
806             [r.model_dump() for r in val_results],
807         )
808         response = {
809             "video_id": video_id,
810             "status": "failed",
811             "message": _validation_failure_message(failures),
812             "results": [r.model_dump() for r in val_results],
813         }
814         return response
815
816     # 2. Run segmenter & indexer
817     logger.info("[API] Video passed checks. Segmenting and indexing...")
818     db.add_video(
819         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
820     )
821
822     segmenter_cls = segmenter_registry.get(seg_name)
823     if not segmenter_cls:
824         # Submit-time validation already 400s unknown names; this is the defensive
825         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
826         available = list(segmenter_registry.list_available().keys())
827         response = {
828             "video_id": video_id,
829             "status": "failed",
830             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
831             "results": [r.model_dump() for r in val_results],
832             "play_segments": [],
833         }
834         return response
835
836     logger.info(f"[API] Using segmenter: {seg_name}")
837     notify(f"segmenting ({seg_name})")
838     segmenter_actual = seg_name
839     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only

```

```

840         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
841         play_wm, cand_wm = db.segment_watermarks(video_id)
842         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
843         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
844         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
845         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
846         # they're dropped on the cloud path in this slice ? a documented follow-up.)
847         remote = _maybe_dispatch_remote(
848             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
849         )
850         if remote is not None:
851             response, segmentation_ran = (
852                 remote # `ran` = did the cloud job actually execute (report)
853             )
854             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
855         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
856         compute_actual = "in_process"
857         segmenter = segmenter_cls(db, config)
858         try:
859             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
860             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
861             with _LOCAL_COMPUTE_LANE:
862                 result = segmenter.process_video(
863                     video_id, local_video, req.player_pool, req.max_frames
864                 )
865             except Exception:
866                 # A raising segmenter must not leave half-written rows: restore the pre-run
867                 # state (same destroy-after-confirm contract as the Failure branch), then
let
868                 # the job worker record the failure.
869                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
870                 raise
871             segmentation_ran = True
872
873         if isinstance(result, Failure):
874             logger.error(f"[API] Segmenter failed: {result.message}")
875             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
876             response = {
877                 "video_id": video_id,
878                 "status": "failed",
879                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
880                 "results": [r.model_dump() for r in val_results],
881                 "play_segments": [],
882             }
883             return response
884
885         segments = result.value if hasattr(result, "value") else result
886         notify("finalizing")
887         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
888
889         response = {
890             "video_id": video_id,
891             "status": "success",
892             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
.value

```

```

893         "results": [r.model_dump() for r in val_results],
894         "play_segments": segments,
895     }
896     return response
897 finally:
898     # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
899     # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
900     # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
901     # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
902     if isinstance(response, dict):
903         response.setdefault(
904             "compute",
905             {
906                 "path": compute_actual or "not_run",
907                 "requested": getattr(req, "compute", None),
908             },
909         )
910     prov = response.get("compute", {}) if isinstance(response, dict) else {}
911     logger.info(
912         "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
913         video_id,
914         getattr(req, "compute", None),
915         prov.get("path"),
916         prov.get("dispatched"),
917         (response or {}).get("status") if isinstance(response, dict) else None,
918     )
919     # Always emit the per-video observability report (next to the video, or to
920     # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
921     # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
922     # doesn't surface arbitrary run signals.)
923     paths = _main.emit_indexer_report(
924         db=db,
925         video_id=video_id,
926         # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
927         # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
928         # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
929         # still preserved in DB provenance via db.add_video.
930         video_path=local_video,
931         config=_report_config_for_input(config, req.video_path),
932         val_results=val_results,
933         val_context=val_context,
934         segmenter_requested=req.segmenter,
935         segmenter_actual=segmenter_actual,
936         was_reingest=was_reingest,
937         segmentation_ran=segmentation_ran,
938     )
939     if paths and isinstance(response, dict):
940         response["reports"] = paths
941     storage.cleanup_acquired_local_input(
942         req.video_path, local_video, getattr(config, "storage", None)
943     )
944
945
946 def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
947     """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
948

```

```

949         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
950         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
951         to a no-op rather than blocking a job it cannot measure."""
952         try:
953             sc = (
954                 storage_config.model_dump()
955                 if hasattr(storage_config, "model_dump")
956                 else (storage_config or {})
957             )
958             backend = storage.get_storage(input_ref.backend, config=sc)
959             return int(backend.stat(input_ref).size)
960         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
not block
961             logger.debug(
962                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
963             )
964             return None
965
966
967     class _CompileError(Exception):
968         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
969         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
970
971         def __init__(self, status_code: int, detail: str):
972             super().__init__(detail)
973             self.status_code = status_code
974             self.detail = detail
975
976
977     def _resolve_compile(
978         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
979     ):
980         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
981         row could change between submit and run). Verifies the video exists, the requested
segments match
982         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
983         so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
984         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
985         video = db.get_video(video_id)
986         if not video:
987             raise _CompileError(404, "Indexed video record not found.")
988         # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
989         try:
990             out_name = safe_output_filename(output_filename)
991         except ValueError as e:
992             raise _CompileError(400, str(e)) from e
993
994         all_segments = db.query_play_segments(video_id, {})
995         matched = [s for s in all_segments if s["id"] in segment_ids]
996         if not matched:
997             raise _CompileError(400, "No matching play segments found to stitch.")
998
999         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1000         kept = []
1001         for s in matched:
1002             meta = s.get("metadata") or {}

```

```

1003         sc = meta.get("shot_count") if isinstance(meta, dict) else None
1004         try:
1005             if sc is not None and int(sc) < stitching.min_shot_count:
1006                 continue
1007         except (TypeError, ValueError):
1008             pass # unparseable count -> treat as unknown, keep
1009         kept.append(s)
1010     if len(kept) < len(matched):
1011         logger.info(
1012             "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
1013             floor.",
1014             stitching.min_shot_count,
1015             len(matched) - len(kept),
1016             len(matched),
1017         )
1018     if not kept:
1019         raise _CompileError(
1020             400,
1021             f"All {len(matched)} matching segments fall below "
1022             f"stitching.min_shot_count={stitching.min_shot_count}.",
1023         )
1024     return video, kept, out_name
1025
1026 def reel_object_uri(cfg, video_id: str, name: str) -> str:
1027     """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
1028     (#463), or "" when
1029     the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
1030     compile
1031     mirror-upload and the /api/highlights redirect:
1032     ``<output_root>/highlights/<video_id>/<name>``. """
1033     root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1034     if not root or "://" not in root:
1035         return ""
1036     return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1037
1038 def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1039     """The active storage settings dict (per-backend config for the storage layer ? S3
1040     endpoint/region,
1041     GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
1042     default backend
1043     isn't silently constructed as a fresh default. """
1044     sc = getattr(cfg, "storage", None)
1045     return sc.model_dump() if sc is not None else None
1046
1047 def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1048     """A short-lived signed URL for a reel mirrored to the output bucket, so
1049     /api/highlights can
1050     307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
1051     ephemeral local
1052     /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
1053     config for BOTH
1054     the existence check and the signing (extracted here to keep backend/main.py lean ?
1055     P23/P24). """
1056     reel_uri = reel_object_uri(cfg, video_id, name)
1057     if not reel_uri:
1058         return None
1059     settings = _storage_settings(cfg)
1060     if storage.exists(reel_uri, config=settings):
1061         return storage.signed_url(reel_uri, config=settings, method="GET")
1062     return None

```

```

1058     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1059         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
1060         stitch the reel,
1061         and return the {status, filepath, download_url} result the SPA polls for. Returns a
1062         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
1063         stitch error)
1064         so the SPA always gets a clean verdict instead of a worker-crash error."""
1065         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
1066         backend.main
1067         notify = progress or (lambda stage: None)
1068         params = json.loads(job.get("params") or "{}")
1069         video_id = job.get("video_id") or ""
1070         segment_ids = params.get("segment_ids") or []
1071         locale = params.get("locale")
1072
1073         notify("resolving")
1074         try:
1075             video, kept, out_name = _resolve_compile(
1076                 db, cfg, video_id, segment_ids, params.get("output_filename", "")
1077             )
1078         except _CompileError as e:
1079             return {"status": "failed", "message": e.detail, "video_id": video_id}
1080
1081         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1082         output_dir = (
1083             getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1084         )
1085         # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
1086         the job);
1087         # unknown/absent locale keeps the configured default unchanged.
1088         localized_watermark = localize_watermark(stitching.watermark, locale)
1089         compiler = _main.VideoCompiler(
1090             output_dir,
1091             begin_padding=stitching.begin_padding,
1092             end_padding=stitching.end_padding,
1093             watermark=localized_watermark,
1094         )
1095         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1096
1097         notify("stitching")
1098         local_src = ""
1099         try:
1100             # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
1101             path). Resolve to
1102             # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
1103             scratch for a URI (the
1104             # same seam the process path uses); without it, compiling a bucket-ingested
1105             video can't open gs://.
1106             local_src = storage.acquire_local_input(
1107                 video["filepath"], getattr(cfg, "storage", None)
1108             )
1109         except Exception as e:
1110             # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
1111             (for remote
1112             # dispatch) doesn't run multiple compiles at once on the box (see
1113             _LOCAL_COMPUTE_LANE).
1114             with _LOCAL_COMPUTE_LANE:
1115                 out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1116         except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
1117             FAILURE, not a crash
1118             logger.error(
1119                 "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1120             )
1121         return {
1122             "status": "failed",

```

```

1113         "message": f"Highlight stitching failed: {e}",
1114         "video_id": video_id,
1115     }
1116     finally:
1117         if local_src:
1118             storage.cleanup_acquired_local_input(
1119                 video["filepath"], local_src, getattr(cfg, "storage", None)
1120             )
1121
1122     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
1123     # URL ? Cloud Run
1124     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
1125     # copy dies when the
1126     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
1127     # (the endpoint
1128     # still falls back to the local file on the same instance).
1129     reel_uri = reel_object_uri(cfg, video_id, out_name)
1130     if reel_uri:
1131         try:
1132             notify("uploading reel")
1133             # Thread the active storage settings so the configured backend (S3
1134             # endpoint/region, GCS
1135             # project/client) is used, not a fresh default.
1136             storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1137         except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
1138             # already succeeded
1139             logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1140
1141     # `filepath` is server-local (not browser-reachable); also return the download URL
1142     # so the SPA needn't
1143     # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1144     download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1145     notify("finished")
1146     return {
1147         "status": "success",
1148         "filepath": out_path,
1149         "download_url": download_url,
1150         "video_id": video_id,
1151     }

```

11. assistant (2026-07-07T15:19:42.214Z)

Now let me examine the actual runners to verify the finding's claims about stub/WSL/native runner selection. Let me find the detector and segmenter files in the real repo.

12. user (2026-07-07T15:19:44.842Z)

```

__init__.py
__pycache__
audio_features.py
base.py
court_gate.py
device.py
fusion_features.py
native_wasb_runner.py
person_detector.py
rally_gate.py
stub_runner.py
tracknet_runner.py
wasb_infer.py
wasb_runner.py
===SEGMENTERS===
__init__.py
__pycache__

```

```

_keyhint.py
base.py
fusion_hybrid.py
gemini.py
motion.py
tracknet_hybrid.py
trajectory_hybrid.py
wasb_hybrid.py
yolo_hybrid.py
===GREP detector_impl===
backend/compute/cloud_run.py:59:     "indexing.detector_impl=native",
backend/config/models.py:87:     # --- Linux-native runner only (detector_impl='native'); ignored
by the WSL runner. ---
backend/config/models.py:209:     # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
backend/config/models.py:210:     detector_impl: str = "auto"
backend/config/models.py:463:     (``indexing.detector_impl`` / ``skip_ai_handoff``,
``storage.backend``) in the loader.
backend/config/presets.py:40:# ``indexing.{detector_impl,skip_ai_handoff}`` and
``storage.backend``. Later milestones extend
backend/config/presets.py:46:         # detector_impl stays "auto" (the runner picks WSL vs
native by env on the GPU box).
backend/config/presets.py:58:         "detector_impl": "native",
backend/config/presets.py:77:         "indexing": {"detector_impl": "stub", "skip_ai_handoff":
True},
backend/config/startup.py:201:     detector_impl = getattr(indexing, "detector_impl", "auto")
backend/config/startup.py:301:         if detector_impl != "stub":
backend/config/startup.py:306:             f"profile 'cpu-mock' normally uses
detector_impl='stub', but detector_impl="
backend/config/startup.py:307:             f"'{detector_impl}'. A real detector defeats
the GPU-free mock.",
backend/pipeline/detectors/native_wasb_runner.py:23:the windowing brain are unchanged. Selected
via the existing ``detector_impl`` seam
backend/pipeline/detectors/native_wasb_runner.py:24:(``trajectory_hybrid._resolve_runner`` ?
``_build_native_runner``); ``detector_impl="native"``.
backend/pipeline/detectors/stub_runner.py:20:``indexing.detector_impl="stub"`` or the
``RALLY_DETECTOR_IMPL=stub`` env var; the real
backend/pipeline/segmenters/fusion_hybrid.py:95:         "detector_impl='native' supports
the WASB substrate only ? set "
backend/pipeline/segmenters/fusion_hybrid.py:106:         "detector_impl": "native",
backend/pipeline/segmenters/trajectory_hybrid.py:71:         f"detector_impl='native' is not
supported for the "
backend/pipeline/segmenters/trajectory_hybrid.py:73:         f"Linux-native runner. Use
detector_impl='auto' (WSL) or 'stub'."
backend/pipeline/segmenters/trajectory_hybrid.py:81:         ``RALLY_DETECTOR_IMPL`` env var
(takes precedence) or ``indexing.detector_impl``:
backend/pipeline/segmenters/trajectory_hybrid.py:93:
os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
backend/pipeline/segmenters/trajectory_hybrid.py:102:         "%s: detector_impl=stub ?
CPU mock runner (no GPU/WSL).",
backend/pipeline/segmenters/trajectory_hybrid.py:106:         "detector_impl": "stub"
backend/pipeline/segmenters/trajectory_hybrid.py:110:         "%s: detector_impl=native ?
Linux-native runner (no WSL).",
backend/pipeline/segmenters/trajectory_hybrid.py:244:         # _resolve_runner honours the CPU-
stub override (RALLY_DETECTOR_IMPL /
backend/pipeline/segmenters/trajectory_hybrid.py:245:         # indexing.detector_impl="stub") so
the pipeline runs GPU/WSL-free; otherwise
backend/pipeline/segmenters/trajectory_hybrid.py:250:         # e.g. detector_impl="native"
for a family without a native runner (tracknet, or
backend/pipeline/segmenters/wasb_hybrid.py:50:         # seam (imported only when
detector_impl='native' is actually selected).
backend/pipeline/segmenters/wasb_hybrid.py:59:         "detector_impl": "native",
backend/worker/__main__.py:9:are byte-for-byte unchanged. Combine with
``RALLY_DETECTOR_IMPL=stub`` and
backend/worker/__main__.py:345:         detector_impl = (

```

```

backend/worker/__main__.py:346:         os.environ.get("RALLY_DETECTOR_IMPL")
backend/worker/__main__.py:347:         or getattr(config.indexing, "detector_impl",
None)
backend/worker/__main__.py:351:         "infer produced 0 segments for video_id=%s
(segmenter=%s, detector_impl=%s). "
backend/worker/__main__.py:361:         detector_impl,
backend/worker/__main__.py:482:         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl:
auto=WSL, native=GPU box,
backend/worker/__main__.py:508:         # e.g. detector_impl='native' for a family without a
native runner (tracknet, or
backend/worker/__main__.py:761:         "(needs videos/ in the corpus; honours detector_impl).",

```

13. assistant (2026-07-07T15:19:55.516Z)

Let me read the runner selection logic and the stub/WSL/native runners to verify the finding's claims.

14. user (2026-07-07T15:19:57.800Z)

```

40     logger = logging.getLogger(__name__)
41
42
43     def _fmt_ts(seconds: float) -> str:
44         seconds = max(0, int(seconds))
45         h, rem = divmod(seconds, 3600)
46         m, s = divmod(rem, 60)
47         return f"{h:02d}:{m:02d}:{s:02d}"
48
49
50     class TrajectoryHybridSegmenter(BasePlaySegmenter):
51         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
52
53         #: config section under `indexing` (velocity_threshold lives there), the
54         #: output subdir for trajectories, and the metadata detector-tag prefix.
55         family: str = ""
56         #: human-readable label used in log lines.
57         display_label: str = ""
58
59         def _build_runner(
60             self, idx_cfg: IndexingConfig
61         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
62             """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
63             raise NotImplementedError
64
65         def _build_native_runner(
66             self, idx_cfg: IndexingConfig
67         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
68             """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
69             native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
70             raise NotImplementedError(
71                 f"detector_impl='native' is not supported for the "
72                 f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
73                 f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
74             )
75
76         def _resolve_runner(
77             self, idx_cfg: IndexingConfig
78         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
79             """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.

```

```

80
81     ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
82     ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
83     is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
84     in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
85     ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
86     (the WSL runner) so the default production path is unchanged
87     (``docs/DECOUPLED_COMPUTE/``).
88     ""
89     # Safety net: accept raw dicts from legacy / test callers.
90     if isinstance(idx_cfg, dict):
91         idx_cfg = IndexingConfig(**idx_cfg)
92     impl = (
93         os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
94     ).lower()
95     if impl == "stub":
96         from backend.pipeline.detectors.stub_runner import (
97             StubConfig,
98             StubDetectorRunner,
99         )
100
101         logger.info(
102             "%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
103             self.display_label or "trajectory",
104         )
105         return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg), {
106             "detector_impl": "stub"
107         })
108     if impl in ("native", "native_wasb", "wasb_native"):
109         logger.info(
110             "%s: detector_impl=native ? Linux-native runner (no WSL).",
111             self.display_label or "trajectory",
112         )
113         return self._build_native_runner(idx_cfg)
114     return self._build_runner(idx_cfg)
115
116 @staticmethod
117 def _probe_fps_width(video_path: str) -> tuple:
118     """Return (fps, frame_width) for a video, with sensible fallbacks."""
119     cap = cv2.VideoCapture(video_path)
120     fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
121     width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
122     cap.release()
123     return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
124
125 @staticmethod
126 def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
127     """Provider-reported exchange count, else a duration-based estimate.
128
129     One canonical implementation ? the twins had drifted (wasb relied on
130     ``int(None)`` raising into the fallback; same result, by accident).
131     """
132     shot_count = segment.get("shuttle_exchange_count")
133     if shot_count is None:
134         return max(3, int(duration / 0.8))
135     try:
136         return int(shot_count)
137     except (ValueError, TypeError):
138         return max(3, int(duration / 0.8))
139
140 # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)

```

```

-----
141     def _enrich_candidate_stats(
142         self,
143         cw: Dict[str, Any],
144         points: List[Any],
145         fps: float,
146         frame_width: float,
147         video_path: str,
148         idx_cfg: "IndexingConfig",
149     ) -> Dict[str, Any]:
150         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
151         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
152         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
153         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
154         return {
155             "peak_velocity": cw["peak_velocity"],
156             "mean_velocity": cw["mean_velocity"],
157             "active_frame_count": cw["active_frame_count"],
158             "track_id_count": cw["track_id_count"],
159         }
160
161     def _select_for_handoff(
162         self,
163         windows: List[Dict[str, Any]],
164         window_stats: List[Dict[str, Any]],
165         idx_cfg: "IndexingConfig",
166     ) -> List[int]:
167         """Indices of the candidate windows that proceed to the AI handoff (and thus may
168         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
169         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.

```

15. user (2026-07-07T15:19:57.995Z)

```

1     """CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".
2
3     A ``DetectorRunner`` that produces a deterministic ``Frame,Visibility,X,Y`` trajectory
4     CSV with no GPU, no WSL, no model weights ? so the whole trajectory-hybrid pipeline
5     (``wasb_hybrid`` / ``tracknet_hybrid``) can run end-to-end on a CPU-only box (CI, a CPU
6     container, a laptop) for regression testing. The person/yolo path already CPU-falls-back
7     (``use_gpu=False``); this closes the last GPU/WSL-only gap so the entire pipeline is
8     runnable ? and regression-gateable ? without a GPU.
9
10    Two modes:
11    * replay ? emit an existing ``Frame,Visibility,X,Y`` CSV verbatim (drive the
12    pipeline with a committed/golden trajectory; deterministic + cross-OS).
13    * synth (default) ? generate a deterministic synthetic trajectory of
14    ``num_rallies`` in-flight bursts (visible + fast-moving frames) separated by dead
15    gaps, so windowing yields a stable, known number of candidate windows.
16
17    This is the CPU seam the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
18    "Optional Linux-native path") and the seed of the future ``LinuxNativeRunner``**
19    (M3.5, ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``). It is selected via
20    ``indexing.detector_impl="stub"`` or the ``RALLY_DETECTOR_IMPL=stub`` env var; the real
21    WSL runners stay the default, so production behaviour is unchanged.
22    """
23
24    import csv as _csv
25    import logging
26    import os
27    import shutil
28    from dataclasses import dataclass
29    from typing import Any, List, Optional, Tuple

```

```

30
31 from backend.pipeline.detectors.base import DetectorRunner
32
33 # Reuse the model-agnostic CSV parsing + windowing (no torch/tf pulled here).
34 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
35
36 logger = logging.getLogger(__name__)
37
38
39 @dataclass
40 class StubConfig:
41     """Settings for the CPU stub runner (built from ``config.json`` ->
42     ``indexing.stub``)."""
43
44     fps: float = 30.0
45     frame_width: float = 1280.0
46     num_rallies: int = 2
47     rally_seconds: float = 4.0
48     gap_seconds: float = (
49         3.0 # dead gap between rallies (> merge_gap so windows don't merge)
50     )
51     lead_seconds: float = 2.0 # dead lead-in
52     step_px: float = (
53         12.0 # per-frame shuttle displacement during a rally (>> velocity_thresh)
54     )
55     replay_csv: Optional[str] = (
56         None # if set + exists, emit this CSV verbatim instead of synthesising
57     )
58
59 @classmethod
60 def from_indexing_cfg(cls, idx_cfg: Any) -> "StubConfig":
61     # Stub config has no corresponding section in IndexingConfig ? it's
62     # test-only. Accept IndexingConfig (typed path) or dict (test compat).
63     s: dict = {}
64     if isinstance(idx_cfg, dict):
65         s = idx_cfg.get("stub", {}) or {}
66     # IndexingConfig has no 'stub' field ? use defaults.
67     return cls(
68         fps=float(s.get("fps", 30.0)),
69         frame_width=float(s.get("frame_width", 1280.0)),
70         num_rallies=int(s.get("num_rallies", 2)),
71         rally_seconds=float(s.get("rally_seconds", 4.0)),
72         gap_seconds=float(s.get("gap_seconds", 3.0)),
73         lead_seconds=float(s.get("lead_seconds", 2.0)),
74         step_px=float(s.get("step_px", 12.0)),
75         replay_csv=s.get("replay_csv"),
76     )
77
78 class StubDetectorRunner(DetectorRunner):
79     """Deterministic CPU trajectory runner (no GPU/WSL). See the module docstring."""
80
81     def __init__(self, cfg: Optional[StubConfig] = None):
82         self.cfg = cfg or StubConfig()
83
84     def healthcheck(self) -> Tuple[bool, str]:
85         return True, "stub CPU detector (no GPU/WSL)"
86
87     def _rows(self) -> List[Tuple[int, int, float, float]]:
88         """Deterministic ``(frame, visible, x, y)`` rows: ``num_rallies`` in-flight
89         bursts
90         separated by dead gaps. During a rally the shuttle oscillates ``step_px``/frame
91         ?
92         well above the default 0.02 velocity threshold ? so every rally frame is
93         "active"

```

```

91         and each burst becomes exactly one candidate window."""
92         cfg = self.cfg
93         rows: List[Tuple[int, int, float, float]] = []
94         frame = 0
95         y = cfg.frame_width / 4.0
96         base_x = cfg.frame_width / 2.0
97         for _ in range(int(cfg.lead_seconds * cfg.fps)):
98             rows.append((frame, 0, 0.0, 0.0))
99             frame += 1
100         for r in range(cfg.num_rallies):
101             for i in range(int(cfg.rally_seconds * cfg.fps)):
102                 x = base_x + (cfg.step_px if i % 2 else -cfg.step_px)
103                 rows.append((frame, 1, x, y))
104                 frame += 1
105             if r != cfg.num_rallies - 1:
106                 for _ in range(int(cfg.gap_seconds * cfg.fps)):
107                     rows.append((frame, 0, 0.0, 0.0))
108                     frame += 1
109         return rows
110
111     def run_predict(
112         self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
113     ) -> Optional[str]:
114         """Write a deterministic trajectory CSV and return its path (never touches the
GPU/WSL)."""
115         os.makedirs(output_win_dir, exist_ok=True)
116         stem = os.path.splitext(
117             os.path.basename(str(video_win_path).replace("\\", "/"))
118         )[0]
119         key = f"{stem}_{video_id[:12]}" if video_id else (stem or "stub")
120         out_csv = os.path.join(output_win_dir, f"{key}_stub.csv")
121
122         if self.cfg.replay_csv and os.path.exists(self.cfg.replay_csv):
123             shutil.copyfile(self.cfg.replay_csv, out_csv)
124             logger.info(
125                 "Stub runner: replayed trajectory %s -> %s",
126                 self.cfg.replay_csv,
127                 out_csv,
128             )
129             return out_csv
130
131         rows = self._rows()
132         with open(out_csv, "w", newline="") as f:
133             w = _csv.writer(f)
134             w.writerow(["Frame", "Visibility", "X", "Y"])
135             for frame, vis, x, y in rows:
136                 w.writerow([frame, vis, f"{x:.3f}", f"{y:.3f}"])
137             logger.info(
138                 "Stub runner: wrote %d synthetic trajectory rows -> %s", len(rows), out_csv
139             )
140             return out_csv
141
142         # Model-agnostic CSV parsing + windowing ? identical to the real runners.
143         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
144         trajectory_to_action_windows = staticmethod(
145             TrackNetRunner.trajectory_to_action_windows
146         )
147
148     def synth_trajectory_from_labels(
149         labels_csv: str,
150         out_csv: str,
151         *,
152         fps: float = 30.0,
153         frame_width: float = 1280.0,

```

```

155         tail_pad: float = 2.0,
156         step_px: float = 12.0,
157     ) -> str:
158         """CPU "mock GPU" for TRAINING: write a deterministic Frame,Visibility,X,Y
trajectory whose
159         in-flight (visible + oscillating) frames cover each labeled rally window and whose
dead frames
160         cover the gaps ? so the synthetic trajectory *encodes* the rally labels.
161
162         This lets the Gen-0 harness train + leave-one-video-out eval end-to-end on coherent
data with
163         no GPU/WSL, validating the whole training pipeline before the real WASB detector
lands. (The
164         learned model is meaningful for plumbing, not for production ? the trajectory is
synthetic.)
165         """
166         rallies: List[Tuple[float, float]] = []
167         with open(labels_csv, newline="", encoding="utf-8") as f:
168             for row in _csv.DictReader(f):
169                 try:
170                     rallies.append((float(row["start_time"]), float(row["end_time"])))
171                 except (KeyError, ValueError):
172                     continue
173         duration = (max(e for _, e in rallies) + tail_pad) if rallies else 10.0
174         nframes = max(1, int(duration * fps))
175         base_x = frame_width / 2.0
176         y = frame_width / 4.0
177         os.makedirs(os.path.dirname(out_csv) or ".", exist_ok=True)
178         with open(out_csv, "w", newline="", encoding="utf-8") as f:
179             w = _csv.writer(f)
180             w.writerow(["Frame", "Visibility", "X", "Y"])
181             for i in range(nframes):
182                 t = i / fps
183                 if any(s <= t < e for s, e in rallies):
184                     x = base_x + (step_px if i % 2 else -step_px)
185                     w.writerow([i, 1, f"{x:.3f}", f"{y:.3f}"])
186                 else:
187                     w.writerow([i, 0, "0.000", "0.000"])
188         return out_csv
189

```

16. assistant (2026-07-07T15:21:06.964Z)

The stub runner's healthcheck does return True unconditionally with no weights. Now let me check the presets, startup validation, and the WSL/native runners to see whether these config combinations validly co-occur and how weights actually resolve.

17. user (2026-07-07T15:21:07.066Z)

```

1         """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3         A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4         config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5         knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7         Precedence (lowest ? highest), realised by the loader::
8
9         built-in model defaults < profile preset < config.json < env < worker --set
10
11         This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12         module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13         (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14         profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==

```

```

15     today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16     spend is never entered silently).
17     """
18
19     from __future__ import annotations
20
21     import copy
22     import os
23     from typing import Any, Mapping, Optional
24
25     # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26     # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27     PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28     COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30     # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31     # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32     COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33         "truly-local": "local_gpu",
34         "cloud-serving": "cloud_run",
35         "cpu-mock": "cpu_mock",
36     }
37
38     # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39     # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40     # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41     # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42     # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43     PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44         "truly-local": {
45             "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46             # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47             # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48             "indexing": {"skip_ai_handoff": True},
49             "storage": {"backend": "local"},
50         },
51         "cloud-serving": {
52             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53             # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54             # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
L4
55             # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56             # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
57             "indexing": {
58                 "detector_impl": "native",
59                 "skip_ai_handoff": False,
60                 # GPU-resident NVDEC decode (#506/#507): validated on a real L4 (golden gate
PASS,
61                 # F1 0.574 ? baseline 0.582 @ SERVED) AND on the live Cloud Run L4 (the
served
62                 # trajectory reproduced the VM run ? 26479 ? 26481 visible on GX010128),
then rolled
63                 # out + canaried (DEPLOY_REPORT_cloud-serving-gpu-resident-
nvdec_2026-07-07). This makes
64                 # nvdec the code-level serving default so it survives an image rebuild; the

```

```

worker's
65         # WASB_DECODE_BACKEND env was the reversible rollout lever. cuda-only
(validator-enforced).
66         "wasb": {
67             "batch_size": 64,
68             "prefetch_batches": 4,
69             "decode_backend": "nvdec_resident",
70         },
71     },
72     "storage": {"backend": "gcs"},
73 },
74 "cpu-mock": {
75     "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
76     # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
77     "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
78     "storage": {"backend": "local"},
79 },
80 }
81
82
83 def is_known_profile(profile: str) -> bool:
84     """True for a non-empty, recognised profile name."""
85     return bool(profile) and profile in PROFILE_PRESETS
86
87
88 def preset_for(profile: str) -> dict[str, Any]:
89     """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
90     module-level table), or ``{}`` for ``""`` / an unknown profile."""
91     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
92
93
94 def _truthy(val: Optional[str]) -> bool:
95     """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
96     if val is None:
97         return False
98     return val.strip().lower() in {"1", "true", "yes", "on"}
99
100
101 def _cuda_available() -> bool:
102     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
103     ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
104     try:
105         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
106         # and optional). Only reached when a caller opts into probe_cuda.
107         import torch
108
109         return bool(torch.cuda.is_available())
110     except Exception:
111         return False
112
113
114 def probe_cuda_available() -> bool:
115     """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
116
117     Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
118     that *wants* the GPU heads-up can pass the result in without the startup module
importing
119     torch itself. Best-effort by design: if torch lives only in a detector subprocess
env

```

```

120         (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
121         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
122         error that could false-fail."""
123         return _cuda_available()
124
125
126     def suggest_profile(
127         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
128     ) -> str:
129         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
130
131         Signals, in priority order:
132
133         - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
134         - ``CI`` truthy                             ? ``"cpu-mock"``
135         - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
136         - otherwise                                 ? ``""`` (no strong suggestion)
137
138         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
139         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
140         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
141         """
142         e = os.environ if env is None else env
143         if e.get("K_SERVICE"):
144             return "cloud-serving"
145         if _truthy(e.get("CI")):
146             return "cpu-mock"
147         if probe_cuda and _cuda_available():
148             return "truly-local"
149         return ""
150

```

18. user (2026-07-07T15:21:07.141Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4     1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5     2. copies wasb_infer.py into the WASB repo's src/ dir,
6     3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7     4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12
13    import logging
14    import os
15    import shlex
16    import subprocess
17    from dataclasses import dataclass
18    from typing import Optional, Tuple
19
20    from backend.config.models import IndexingConfig
21    from backend.pipeline.detectors.base import (
22        DetectorRunner,
23        WslCommandMixin,
24        wsl_tilde_quote,
25    )
26

```

```

27     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
28     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
29     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, to_wsl_mnt_path
30
31     logger = logging.getLogger(__name__)
32
33     # Windows path to the inference wrapper we stage into WSL.
34     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
35
36
37     @dataclass
38     class WasbConfig:
39         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
40
41         distro: str = "Ubuntu"
42         repo_dir: str = "~/models/WASB-SBDT"
43         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
44         conda_env: str = "wasb"
45         weights_path: str = (
46             "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
47         )
48         sport: str = "badminton"
49         wsl_stage_dir: str = "~/clips"
50         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
51         keep_frames: bool = (
52             False # retain staged video + frame cache after success (debug only)
53         )
54         min_free_gb: float = 0.0 # absolute floor (GB) atop the ~15x-source BLOCK guard (0
= off)
55
56         # FAST PATH (default OFF until verified): decode the staged video on the fly inside
57         # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
58         # extraction that dominates a long clip. Output is bit-identical to the PNG path
59         # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
60
61         stream_video: bool = False
62
63         @classmethod
64         def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "WasbConfig":
65             if isinstance(idx_cfg, dict):
66                 idx_cfg = IndexingConfig(**idx_cfg)
67             w = idx_cfg.wasb
68             return cls(
69                 distro=w.wsl_distro,
70                 repo_dir=w.repo_dir,
71                 conda_sh=w.conda_sh,
72                 conda_env=w.conda_env,
73                 weights_path=w.weights_path,
74                 sport=w.sport,
75                 wsl_stage_dir=w.wsl_stage_dir,
76                 timeout_sec=w.timeout_sec,
77                 keep_frames=w.keep_frames,
78                 min_free_gb=w.min_free_gb,
79                 stream_video=w.stream_video if w.stream_video is not None else False,
80             )
81
82         class WasbRunner(WslCommandMixin, DetectorRunner):
83             def __init__(self, cfg: WasbConfig):
84                 self.cfg = cfg
85
86             def healthcheck(self) -> Tuple[bool, str]:
87                 """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
88                 cmd = (

```

```

88         f"conda activate {self.cfg.conda_env} && "
89         f"python -c \"import torch; print('torch', torch.__version__, "
90         f"'cuda', torch.cuda.is_available())\""
91     )
92     try:
93         res = self._wsl_bash(cmd)
94     except FileNotFoundError:
95         return False, "`wsl` not found ? Windows + WSL2 required."
96     except subprocess.TimeoutExpired:
97         return False, "WSL healthcheck timed out."
98     out = (res.stdout or "").strip()
99     if res.returncode != 0:
100         return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
101     return True, out
102
103 def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
104     src_mnt = to_wsl_mnt_path(video_win_path)
105     reason = self.wsl_source_unreadable_reason(src_mnt)
106     if reason:
107         logger.error("Cannot stage video into WSL: %s", reason)
108         return None
109     dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
110     cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
111     {wsl_tilde_quote(dest)} && echo OK"
112     try:
113         res = self._wsl_bash(cmd)
114     except subprocess.TimeoutExpired:
115         logger.error("Staging video into WSL timed out.")
116         return None
117     if res.returncode != 0 or "OK" not in (res.stdout or ""):
118         logger.error(
119             "Failed to stage video into WSL: %s", (res.stderr or res.stdout).strip()
120         )
121         return None
122     return dest
123
124 def _stage_infer_script(self) -> bool:
125     """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
126     src_mnt = to_wsl_mnt_path(_INFER_WIN)
127     cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo"
128     OK"
129     res = self._wsl_bash(cmd)
130     return res.returncode == 0 and "OK" in (res.stdout or "")
131
132 def run_predict(
133     self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
134 ) -> Optional[str]:
135     """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
136
137     All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
138     two videos that share a filename (e.g. two ``match.mp4`` from different folders)
139     can never reuse each other's staged frames, resumable cache, or CSV.
140
141     # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
142     # translation: to_wsl_mnt_path passes relative paths through unchanged, so
143     # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
144     # the CSV landed inside WSL while Windows polled the repo-relative path.
145     output_win_dir = os.path.abspath(output_win_dir)
146     os.makedirs(output_win_dir, exist_ok=True)
147     # Normalize for cross-platform basename (tests use Windows paths on Linux).
148     video_win_path = str(video_win_path).replace("\\", "/")
149     base = os.path.basename(video_win_path)
150     stem, ext = os.path.splitext(base)
151     # Unique, content-derived key: same filename + different bytes -> different key.
152     key = f"{stem}__{video_id[:12]}" if video_id else stem

```

```

151 wsl_out_dir = to_wsl_mnt_path(output_win_dir)
152 expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
153
154 if not self._stage_infer_script():
155     logger.error("Could not stage wasb_infer.py into WSL.")
156     return None
157 staged_video = self._stage_video(video_win_path, f"{key}{ext}")
158 if staged_video is None:
159     return None
160 frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
161
162 # Disk guard (D11): a 4K clip can extract ~100 GB of PNG frames. BLOCK before
extraction if
163 # the WSL stage fs can't hold ~15× the source ? a mid-run disk-fill wastes a
long run and
164 # looks like a quality fail. Best-effort measuring (an undeterminable read never
blocks).
165 # Streaming never materializes the PNGs, so the guard only matters off the fast
path.
166 if not self.cfg.stream_video:
167     shortfall = self._wsl_frame_stage_shortfall(video_win_path)
168     if shortfall:
169         logger.error("WASB inference aborted ? %s", shortfall)
170         return None
171
172 # Fast path: stream-decode the video in-process (no PNG extraction). Output is
173 # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
174 if self.cfg.stream_video:
175     frame_args = "--stream-video "
176 else:
177     frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
178 cmd = (
179     f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
180     f"python wasb_infer.py "
181     f"--video {wsl_tilde_quote(staged_video)} "
182     f"{frame_args}"
183     f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
184     f"--sport {shlex.quote(self.cfg.sport)} "
185     f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}}"
186 )
187 logger.info("Running WASB inference on %s ...", base)
188 try:
189     res = self._wsl_bash(cmd)
190 except subprocess.TimeoutExpired:
191     logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
192     return None
193 if res.returncode != 0:
194     logger.error(
195         "WASB inference failed:\n%s", (res.stderr or res.stdout)[-2000:]
196     )
197     return None
198 if not os.path.exists(expected_csv_win):
199     logger.error(
200         "WASB finished but expected CSV not found at %s", expected_csv_win
201     )
202     return None
203 logger.info("WASB trajectory CSV: %s", expected_csv_win)
204
205 # Success ? the trajectory CSV is the durable output; the staged video copy and
206 # the (huge) decoded frame cache are now pure intermediates. Delete them unless
207 # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
208 # so the frame cache + detector cache remain for a resumable re-run.
209 # (Streaming never writes a frame cache, so there's only the staged video to
drop.)

```

```

210         if not self.cfg.keep_frames:
211             stale = (
212                 [staged_video] if self.cfg.stream_video else [frames_dir, staged_video]
213             )
214             logger.info(
215                 "Cache hygiene: removing staged video%s for %s "
216                 "(set indexing.wasb.keep_frames=true to retain).",
217                 "" if self.cfg.stream_video else " + frame cache",
218                 key,
219             )
220             self._wsl_rm_rf(stale)
221             return expected_csv_win
222
223     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
224     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
225     trajectory_to_action_windows = staticmethod(
226         TrackNetRunner.trajectory_to_action_windows
227     )
228

```

19. assistant (2026-07-07T15:23:43.630Z)

The stub healthcheck returns True unconditionally, and the WSL runner reads weights inside WSL via ``wsl_tilde_quote(cfg.weights_path)`` and never consults ``WASB_WEIGHTS``. Crucially, the presets show ``detector_impl=stub`` pairs with ``compute_target=cpu_mock`` (not `cloud_run`) and ``detector_impl=native`` pairs with ``cloud_run``. Let me verify the native runner's weight resolution and the config model, then concretely test the guard.

20. user (2026-07-07T15:23:45.266Z)

```

1     """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3     The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4     "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5     It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6     Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7     that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8     see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10    * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11    ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12    (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13    * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14    (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15    strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16    * weights / repo / python / scratch come from the environment first
17    (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18    ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20    It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__vid12_wasb.csv``
21    name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22    ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23    the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24    (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26    Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
runner

```

```

27     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28     candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31     vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32     artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34     not the numbers.
35     """
36
37     import logging
38     import os
39     import shutil
40     import subprocess
41     from dataclasses import dataclass
42     from typing import List, Optional, Tuple
43
44     from backend.config.models import IndexingConfig
45     from backend.pipeline.detectors.base import DetectorRunner
46     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51     logger = logging.getLogger(__name__)
52
53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (
60         "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61     )
62
63     # Decode backends wasb_infer.py understands (#506). Mirrored by its --decode-backend
choices;
64     # healthcheck rejects anything else up front so a typo can't reach argparse as rc=2 mid-
run.
65     _VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")
66
67     # Probe for the GPU-resident decode dep (#506): a wasb env without PyNvVideoCodec must
fail the
68     # healthcheck, not the paid inference subprocess. (The NVDEC driver libs ? libnvcuid ?
are only
69     # exercised at decoder INIT, so a missing lib still surfaces in the subprocess; this
catches the
70     # common misconfig, a wasb env built without the package.)
71     _PYNVC_PROBE_CODE = "import PyNvVideoCodec"
72
73
74     @dataclass
75     class NativeWasbConfig:
76         """Resolved settings for a Linux-native WASB run.
77
78         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence

```

```

79         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
80         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
81         """
82
83         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
84         python_bin: str = (
85             "python" # the `wasb` conda env python (invoked by path, no activate)
86         )
87         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
88         sport: str = "badminton"
89         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
90         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
91         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
92         keep_frames: bool = (
93             False # retain the decoded frame cache after success (debug only)
94         )
95         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
96         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
97         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
98         stream_video: bool = True
99         # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
100         # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
101         batch_size: int = 0
102         prefetch_batches: int = 0
103         # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend /
WASB_DECODE_BACKEND).
104         # "cpu" (default) = omit the flag ? wasb_infer.py's historical cv2 decode paths,
argv
105         # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
only).
106         decode_backend: str = "cpu"
107
108         @classmethod
109         def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
110             if isinstance(idx_cfg, dict):
111                 idx_cfg = IndexingConfig(**idx_cfg)
112             w = idx_cfg.wasb
113             env = os.environ.get
114             # stream_video is tri-state in config: None/absent => native default (stream);
an
115             # explicit True/False still wins (e.g. False for a container cv2 can't seek).
116             sv = w.stream_video
117             return cls(
118                 repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
119                 python_bin=env("WASB_PYTHON") or w.python_bin,
120                 weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
121                 sport=w.sport,
122                 device=env("WASB_DEVICE") or w.device,
123                 scratch_dir=env("WASB_SCRATCH")
or env("RALLY_SCRATCH")
124                 or env("TMPDIR")
125                 or "",
126                 timeout_sec=w.timeout_sec,
127                 keep_frames=w.keep_frames,
128                 stream_video=(True if sv is None else bool(sv)),
129                 batch_size=int(w.batch_size or 0),
130                 prefetch_batches=int(w.prefetch_batches or 0),
131                 decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
132             )
133

```

```

134
135
136     class NativeWasbRunner(DetectorRunner):
137         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
138
139         def __init__(self, cfg: NativeWasbConfig):
140             self.cfg = cfg
141             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
142             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
143             self._cuda_available: Optional[bool] = None
144
145             # --- low-level native exec (the single place tests patch)
-----
146         def _run(
147             self, argv: List[str], cwd: Optional[str] = None
148         ) -> subprocess.CompletedProcess:
149             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
150             return subprocess.run(
151                 argv,
152                 cwd=cwd,
153                 capture_output=True,
154                 text=True,
155                 timeout=(self.cfg.timeout_sec or None),
156             )
157
158             # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
159         def _probe_cuda(self) -> bool:
160             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
161             (missing python / timeout / import error) is treated as 'no CUDA'."""
162             try:
163                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
164             except (FileNotFoundError, subprocess.TimeoutExpired):
165                 return False
166             return res.returncode == 0 and "cuda True" in (res.stdout or "")
167
168         def _cuda_is_available(self) -> bool:
169             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
170             if self._cuda_available is None:
171                 self._cuda_available = self._probe_cuda()
172             return self._cuda_available
173
174         def _effective_device(self) -> str:
175             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
176             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
177             if self.cfg.device == AUTO:
178                 return DeviceContext(AUTO, self._cuda_is_available()).effective
179             return self.cfg.device
180
181         def _repo_src(self) -> str:
182             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
183
184         def _copy_infer_script(self) -> bool:
185             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
186             repo_src = self._repo_src()
187             try:
188                 os.makedirs(repo_src, exist_ok=True)

```

```

189         shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
190         return True
191     except OSError as e:
192         logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
193         return False
194
195     def _rm_rf(self, path: Optional[str]) -> None:
196         """Best-effort recursive delete of a native path (post-success cleanup; never
197         raises)."""
198         if path:
199             shutil.rmtree(path, ignore_errors=True)
200
201     def healthcheck(self) -> Tuple[bool, str]:
202         """Verify weights + repo present and the `wasb` python imports torch (and sees a
203         GPU on cuda)."""
204         if not self.cfg.weights_path:
205             return False, (
206                 "WASB weights path is empty ? set WASB_WEIGHTS (or
207                 indexing.wasb.weights_path)."
208             )
209         weights = os.path.expanduser(self.cfg.weights_path)
210         if not os.path.exists(weights):
211             return False, f"WASB weights not found at {weights}"
212         repo_src = self._repo_src()
213         if not os.path.isdir(repo_src):
214             return False, (
215                 f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
216                 f"(set WASB_REPO_DIR / indexing.wasb.repo_dir)."
217             )
218         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
219         # beats a cryptic
220         # argparse rc=2 from wasb_infer.py at run time.
221         if self.cfg.device not in VALID_DEVICES:
222             return False, (
223                 f"unknown device {self.cfg.device!r} ? valid: {'',
224                 ''.join(VALID_DEVICES)})."
225             )
226         if self.cfg.decode_backend not in _VALID_DECODE_BACKENDS:
227             return False, (
228                 f"unknown decode_backend {self.cfg.decode_backend!r} ? valid: "
229                 f"{'', ''.join(_VALID_DECODE_BACKENDS)})."
230             )
231         try:
232             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
233         except FileNotFoundError:
234             return (
235                 False,
236                 f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
237             )
238         except subprocess.TimeoutExpired:
239             return False, "native torch healthcheck timed out."
240         out = (res.stdout or "").strip()
241         if res.returncode != 0:
242             return (
243                 False,
244                 f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
245                 out).strip()}",
246             )
247         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
248         subprocess).
249         self._cuda_available = "cuda True" in out
250         ctx = DeviceContext(self.cfg.device, self._cuda_available)
251         if ctx.cuda_required_but_missing:
252             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
253             a silent

```

```

246         # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
247         return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
248     if ctx.fell_back:
249         logger.warning(
250             "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
251             "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
252             out,
253         )
254     # NVDEC decode is CUDA-only: an auto?cpu fallback (or explicit cpu/mps, which
the
255     # config validator already rejects) cannot run the GPU-resident path ? fail loud
256     # instead of letting wasb_infer.py die mid-run on a paid box.
257     if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
258         return False, (
259             f"decode_backend=nvdec_resident requires CUDA, but the effective device
is "
260             f"{ctx.effective!r} ({out}) ? NVDEC decode runs on the GPU."
261         )
262     if self.cfg.decode_backend == "nvdec_resident":
263         try:
264             nv = self._run([self.cfg.python_bin, "-c", _PYNVC_PROBE_CODE])
265         except (FileNotFoundError, subprocess.TimeoutExpired):
266             return False, "PyNvVideoCodec probe failed to run (python
missing/timeout)."
267         if nv.returncode != 0:
268             return False, (
269                 f"decode_backend=nvdec_resident but `{_PYNVC_PROBE_CODE}` failed in
"
270                 f"{self.cfg.python_bin!r}: {(nv.stderr or nv.stdout or
''.strip())[-300:]} "
271                 f"? the wasb env needs PyNvVideoCodec (deploy/cloudrun/Dockerfile /
"
272                 f"gpu_setup.sh stage it)."
273             )
274     return True, out
275
276     def run_predict(
277         self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
278     ) -> Optional[str]:
279         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
280
281         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
282         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
283         same downstream contract, so the parity comparison is apples-to-apples.
284         """
285         output_dir = os.path.abspath(output_win_dir)
286         os.makedirs(output_dir, exist_ok=True)
287         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
288         norm = str(video_win_path).replace("\\", "/")
289         stem, _ext = os.path.splitext(os.path.basename(norm))
290         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
291         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
292         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
293
294         if not self._copy_infer_script():
295             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
296             return None
297
298         weights = os.path.expanduser(self.cfg.weights_path)
299         argv = [

```

```

300         self.cfg.python_bin,
301         "wasb_infer.py",
302         "--video",
303         os.path.abspath(str(video_win_path)),
304     ]
305 frames_dir: Optional[str] = None
306 if self.cfg.decode_backend == "nvdec_resident":
307     # GPU-resident path (#506): wasb_infer.py decodes the video on the GPU
308     # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so
309     # --stream-video nor --frames_out_dir applies and there is no frame cache to
310     # clean.
311     argv += ["--decode-backend", self.cfg.decode_backend]
312 elif self.cfg.stream_video:
313     # Fast path: decode in-process, no PNG extraction. Output is bit-identical
314     # to disk.
315     argv.append("--stream-video")
316 else:
317     scratch = self.cfg.scratch_dir or output_dir
318     frames_dir = os.path.join(scratch, f"{key}_frames")
319     argv += ["--frames_out_dir", frames_dir]
320 # M4: resolve device="auto" to the effective device (cuda-if-available-else-
321 # cpu); an
322 # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
323 wasb_infer.py.
324 device = self._effective_device()
325 if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
326     # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu on
327     # a
328     # GPU-less box): NVDEC decode cannot run without CUDA ? fail before the
329     subprocess.
330     logger.error(
331         "decode_backend=nvdec_resident requires CUDA, but the effective device
332         is "
333         "%r ? aborting before the WASB subprocess.",
334         device,
335     )
336     return None
337 argv += [
338     "--weights",
339     weights,
340     "--sport",
341     self.cfg.sport,
342     "--device",
343     device,
344     "--out",
345     out_csv,
346 ]
347 # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
348 # its
349 # byte-parity guarantee) is unchanged when the knobs are unset.
350 if self.cfg.batch_size:
351     argv += ["--batch-size", str(self.cfg.batch_size)]
352 if self.cfg.prefetch_batches and self.cfg.decode_backend != "nvdec_resident":
353     # Prefetch overlaps CPU-side batch assembly with GPU compute ? meaningless
354     for
355     # the GPU-resident decoder (there is no CPU assembly), whose wasb_infer
356     branch
357     # ignores the flag; keep the argv honest instead of passing a dead knob.
358     argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
359
360 logger.info(
361     "Running native WASB inference (device=%s%s) on %s ...",
362     device,

```

```

353         " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
354         os.path.basename(norm),
355     )
356     try:
357         res = self._run(argv, cwd=self._repo_src())
358     except FileNotFoundError:
359         logger.error(
360             "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
361         )
362         return None
363     except subprocess.TimeoutExpired:
364         logger.error(
365             "WASB native inference timed out after %ss.", self.cfg.timeout_sec
366         )
367         return None
368     if res.returncode != 0:
369         logger.error(
370             "WASB native inference failed:\n%s",
371             (res.stderr or res.stdout or "")[-2000:],
372         )
373         return None
374     if not os.path.exists(out_csv):
375         logger.error(
376             "WASB native finished but expected CSV not found at %s", out_csv
377         )
378         return None
379     logger.info("WASB trajectory CSV: %s", out_csv)
380
381     # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
382     # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
383     # (Streaming never materializes frames, so there is nothing to drop on that
path.)
384     if not self.cfg.keep_frames and frames_dir:
385         logger.info(
386             "Cache hygiene: removing frame cache for %s "
387             "(set indexing.wasb.keep_frames=true to retain).",
388             key,
389         )
390         self._rm_rf(frames_dir)
391     return out_csv
392
393     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
394     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
395     trajectory_to_action_windows = staticmethod(
396         TrackNetRunner.trajectory_to_action_windows
397     )
398

```

21. user (2026-07-07T15:23:45.997Z)

```

180     ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ?
GPU checks
181     are skipped; the API server passes ``None``, the worker passes
``presets.probe_cuda_available()``).
182     ``include_exposure`` adds the exposure-safety posture (``_exposure_checks``) ? the
HTTP **server**
183     passes ``True``; the worker leaves it ``False`` (it is never "exposed"). Both
dimensions are
184     DEFAULT-OFF: no profile ? no profile checks; ``edge_auth_enabled=False`` ? no
exposure checks.
185     Pure: no IO (beyond the ``find_spec`` auth probe), no torch import ? the caller owns
the GPU probe.

```

```

186     """
187     e = os.environ if env is None else env
188     checks: list[StartupCheck] = []
189
190     # Exposure-safety posture ? INDEPENDENT of deployment.profile (a tunnel-to-a-box
exposure often
191     # runs with no profile set), gated on edge_auth_enabled (default-OFF). Server-only.
192     if include_exposure:
193         checks.extend(_exposure_checks(config, e))
194
195     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
196     if not profile:
197         return checks # no profile ? only the exposure posture (itself off unless edge
auth is on).
198
199     storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
200     indexing = getattr(config, "indexing", None)
201     detector_impl = getattr(indexing, "detector_impl", "auto")
202     device = getattr(getattr(indexing, "wasb", None), "device", "cuda")
203
204     if profile == "truly-local":
205         # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud
backend here is
206         # an inconsistent (but not fatal) choice worth surfacing.
207         if storage_backend not in ("local", "gdrive"):
208             checks.append(
209                 StartupCheck(
210                     LEVEL_WARN,
211                     "STORAGE_UNUSUAL",
212                     f"profile 'truly-local' usually uses local/gdrive storage, but
storage_backend="
213                     f"'{storage_backend}'. Reads will go to a remote backend.",
214                 )
215             )
216         # GPU heads-up (warning only ? the detector healthcheck is the authority; the
probe is
217         # best-effort and may be False even when a subprocess detector env has CUDA).
218         if cuda_available is False:
219             if device == "cuda":
220                 checks.append(
221                     StartupCheck(
222                         LEVEL_WARN,
223                         "CUDA_NOT_VISIBLE",
224                         "device='cuda' but no CUDA GPU is visible to this process. If
the box truly "
225                         "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
226                         "'auto' for the CPU fallback (M4).",
227                     )
228                 )
229             elif device == "auto":
230                 checks.append(
231                     StartupCheck(
232                         LEVEL_WARN,
233                         "CPU_FALLBACK",
234                         "device='auto' and no CUDA GPU is visible to THIS process ? auto
may resolve "
235                         "to CPU (much slower). If torch lives only in the detector
subprocess env this "
236                         "can be a false alarm; the runner's DeviceContext/healthcheck is
the authority.",
237                     )
238                 )
239
240     elif profile == "cloud-serving":

```

```

241         # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
242         if storage_backend not in _CLOUD_STORAGE:
243             checks.append(
244                 StartupCheck(
245                     LEVEL_ERROR,
246                     "STORAGE_INCOHERENT",
247                     f"profile 'cloud-serving' requires a cloud storage backend "
248                     f"('{','.join(_CLOUD_STORAGE)}'), but
storage_backend='{storage_backend}'. "
249                     "A cloud job cannot read/write a local-only backend.",
250                 )
251             )
252         elif storage_backend == "gcs" and not _creds_discoverable(e):
253             checks.append(
254                 StartupCheck(
255                     LEVEL_WARN,
256                     "CREDS_UNCONFIRMED",
257                     "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying
on Application "
258                     "Default Credentials. The GCS backend will fail loudly at init if
ADC can't resolve.",
259                 )
260             )
261         # The AI-handoff key: with skip_ai_handoff=False the segmenter demands
262         # ${AI_PROVIDER}_API_KEY at rally time and bails to an EMPTY result when it is
263         # missing ? a silent 0-rally "success" AFTER the GPU pass is paid for (observed
on
264         # the first real cloud-serving litmus, 2026-07-03). The env cannot appear later
in
265         # a running container, so this is a zero-false-positive boot-time fatal.
266         skip_ai = bool(getattr(indexing, "skip_ai_handoff", False))
267         provider = str(getattr(config, "ai_provider", "gemini") or "gemini")
268         key_var = f"{provider.upper()}_API_KEY"
269         if not skip_ai and not e.get(key_var):
270             checks.append(
271                 StartupCheck(
272                     LEVEL_ERROR,
273                     "AI_HANDOFF_KEY_MISSING",
274                     f"profile 'cloud-serving' runs the {provider} handoff "
275                     f"({indexing.skip_ai_handoff=False}) but {key_var} is not set ? the
segmenter "
276                     f"would bail to an empty result after the paid GPU pass. Mount the
key "
277                     f"(e.g. a Secret Manager secret) or set
indexing.skip_ai_handoff=True for "
278                     "the fully-local candidates-as-rallies mode.",
279                 )
280             )
281         if device == "cpu":
282             checks.append(
283                 StartupCheck(
284                     LEVEL_WARN,
285                     "CLOUD_CPU_DEVICE",
286                     "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
287                     "detection will not use the GPU.",
288                 )
289             )
290         elif cuda_available is False and device in ("cuda", "auto"):
291             checks.append(
292                 StartupCheck(
293                     LEVEL_WARN,
294                     "CLOUD_CUDA_NOT_VISIBLE",
295                     "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to

```

```

this process.",
296         )
297     )
298
299     elif profile == "cpu-mock":
300         # The mock profile is for CI/regression ? a real detector here defeats the
purpose.
301         if detector_impl != "stub":
302             checks.append(
303                 StartupCheck(
304                     LEVEL_WARN,
305                     "MOCK_NONSTUB_DETECTOR",
306                     f"profile 'cpu-mock' normally uses detector_impl='stub', but
detector_impl="
307                     f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
308                 )
309             )
310
311     return checks
312
313
314     def enforce_startup_checks(
315         config: Any,
316         *,
317         cuda_available: Optional[bool] = None,
318         env: Optional[Mapping[str, str]] = None,
319         raise_on_error: bool = True,

```

22. assistant (2026-07-07T15:24:48.457Z)

I now have the full picture on weight resolution. The native runner resolves weights via ``WASB_WEIGHTS`` or `indexing.wasb.weights_path` then ``os.path.exists(os.path.expanduser(...))`` (lines 120, 202-208) ? which is **exactly** what the guard does. So the guard faithfully mirrors the **native** runner, and the ``cloud-serving`` preset (`compute_target=cloud_run`) uses ``detector_impl=native``. Let me concretely verify that ``_cloud_available`` is `False` under the `cpu-mock` and `truly-local` presets (so the guard is skipped there) and only `True` under `cloud-serving`.

23. user (2026-07-07T15:24:51.754Z)

```

195     court_polygon: Optional[list[list[float]]] = None
196     # Net line for the person two-sided-motion split (person features only ? the shuttle
197     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
198     net_axis: Literal["x", "y"] = "y"
199     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
200
201
202     class IndexingConfig(BaseModel):
203         default_segmenter: str = "yolo_hybrid"
204         use_gpu: bool = True
205         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
206         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
207         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
208         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
209         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
210         detector_impl: str = "auto"
211         motion_threshold: float = 0.08
212         min_segment_duration: float = 3.0
213         dead_time_merge_gap: float = 2.0
214         chunk_duration: float = 90.0
215         chunk_overlap: float = 10.0
216         detector_model: str = "fasterrcnn_resnet50_fpn"
217         detector_score_threshold: float = 0.5
218         yolo_velocity_threshold: float = 0.15
219         yolo_frame_skip: int = 3

```

```

220         yolo_tracker: str = "bytetrack.yaml"
221         yolo_prefetch_workers: int = 2
222         min_action_window_duration: float = 2.0
223         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
224         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?)

```

24. user (2026-07-07T15:24:51.781Z)

```

158: def _cloud_available(cfg) -> bool:
=== is the fix already installed? ===
0

```

25. assistant (2026-07-07T15:25:00.486Z)

The installed `orchestration.py` has `\_cloud\_available` but not the fix yet ? good, I can import the real `\_cloud\_available` and presets to concretely test reachability. Let me run a grounded check.

26. user (2026-07-07T15:25:16.160Z)

Exit code 1

Traceback (most recent call last):

```

File "C:\Users\avidu\AppData\Local\Temp\claude\c--Users-avidu-Projects-badminton-highlight-indexer\05c32f87-3bfb-4a7f-861c-0050ac4169ac\scratchpad\verify_guard.py", line 6, in <module>
    from backend.config.models import AppConfig
ModuleNotFoundError: No module named 'backend'

```

27. user (2026-07-07T15:25:27.640Z)

```

profile=truly-local  compute_target=local_gpu  detector_impl=auto  cloud_available=False
guard_fires(wasb_hybrid,local)=False
    default wasb.weights_path = '~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar'  expanduser=exists=False
profile=cloud-serving compute_target=cloud_run  detector_impl=native  cloud_available=True
guard_fires(wasb_hybrid,local)=True
    default wasb.weights_path = '~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar'  expanduser=exists=False
profile=cpu-mock  compute_target=cpu_mock  detector_impl=stub  cloud_available=False
guard_fires(wasb_hybrid,local)=False
    default wasb.weights_path = '~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar'  expanduser=exists=False

```

28. assistant (2026-07-07T15:25:40.354Z)

Concrete confirmation of the key structural fact: under every coherent preset, the guard is either ****correct**** (cloud-serving ? native, which the guard faithfully mirrors) or ****never fires**** (truly-local/WSL and cpu-mock/stub both give `\_cloud\_available=False`). Let me now confirm the finding's mechanism does require an off-preset hand-mixed config.

29. user (2026-07-07T15:25:57.365Z)

```

scenario(a) hand-mixed cloud_run+stub: detector_impl=stub cloud_available=True guard_fires=True
(stub would succeed in-process)
scenario(a) env RALLY_DETECTOR_IMPL=stub on cloud_run box: guard_fires=True (guard ignores the
env override)
=> false-block requires compute_target=cloud_run PAIRED WITH a non-native detector_impl (off-
preset).

```

30. assistant (2026-07-07T15:26:31.686Z)

I have everything I need. My analysis is fully grounded in the real code and concretely verified.

****Summary of verification:****

The finding's *technical* claims are all TRUE against the real code:

- `\_local\_detector\_weights\_present` (orchestration_new.py:166-187) resolves weights via `WASB\_WEIGHTS ? indexing.wasb.weights\_path` + `os.path.exists(os.path.expanduser(...))`, never consulting `detector\_impl`/`RALLY\_DETECTOR\_IMPL`.
- `StubDetectorRunner.healthcheck()` returns `(True, ...)` unconditionally, no weights (stub_runner.py:84-85).
- `WasbRunner` (WSL) reads weights inside WSL via `wsl\_tilde\_quote(cfg.weights\_path)` and never consults `WASB\_WEIGHTS` (wasb_runner.py:183).
- The guard's resolution *exactly* mirrors the **native** runner (native_wasb_runner.py:120, 202-208).

But the claimed *false-block* of a would-succeed run* does NOT occur in any coherent deployment. The guard only fires when `\_cloud\_available(cfg)` is True, and I concretely verified across all three designed presets:

- `cloud-serving` ? compute_target=cloud_run, **detector_impl=native** ? `\_cloud\_available=True`, guard fires and is **correct** (mirrors native faithfully; the exact bug it fixes).
- `truly-local` ? compute_target=local_gpu, auto/WSL ? `\_cloud\_available=False` ? guard **skipped** (WSL runs).
- `cpu-mock` ? compute_target=cpu_mock, stub ? `\_cloud\_available=False` ? guard **skipped** (stub runs).

The stub/WSL false-block is reachable only by hand-mixing `compute\_target=cloud\_run` with a non-native `detector\_impl` ? the per-knob "drift" the preset system explicitly exists to prevent ? and even then, `\_cloud\_available=True` guarantees the "Choose 'Run in cloud'" escape actually works.

31. user (2026-07-07T15:26:47.321Z)

Structured output provided successfully